IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 28 May 2026, accepted 22 June 2026, date of publication 26 June 2026, date of current version 9 July 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3707573

SURVEY

LLM-Based Intelligent Agents for Cybersecurity: A Tutorial and Survey of Automated Vulnerability Discovery

ROBEL ADDIS, KIRAN PORTER, JAYDEN RYAN, MICAH STULL, SHENGJIE XU, (Senior Member, IEEE), PAUL WAGNER, ROBERT HONOMICHL, AND STEVEN FORSYTH

College of Information Science, The University of Arizona, Tucson, AZ 85721, USA

Corresponding author: Shengjie Xu (sjxu@arizona.edu)

This work was supported in part by the Tides Foundation (Prime Sponsor: Google Inc.) "Supporting Communities and Small Business by Conducting Risk and Vulnerability Assessments through Student Experiential Learning Opportunities" under Award ID TF2405-120744, and in part by Arizona Cybersecurity Clinic at The University of Arizona.

ABSTRACT This paper provides a tutorial and survey of LLM-based agents for automated vulnerability discovery and penetration testing. The rapid advancements in Large Language Models (LLMs) have opened new possibilities for their application in cybersecurity. Unlike traditional tools, LLMs can process natural language, retain context, and perform structured reasoning, enabling automation of complex security workflows. By equipping these models with memory mechanisms and tool-use interfaces, researchers have developed LLM-driven autonomous agents capable of performing specialized tasks such as web application testing, source code auditing, fuzzing, and network security assessment. Multi-agent architectures have emerged in which agents assume specialized roles (e.g., distinct penetration-test phases or exploit types). In addition to surveying existing applications, this work provides a step-by-step walkthrough of integrating agentic AI into penetration testing workflows. The walkthrough explores four phases: (I) mission scoping and prompt engineering for test definition and constraint enforcement, (II) autonomous exploration and tool selection for target interaction, (III) vulnerability hypothesis formation and verification through experiment design and feedback, and (IV) payload generation and refinement to transform validated findings into concrete exploits. The survey synthesizes 155 cited sources (2022–early 2026) using explicit inclusion/exclusion criteria, with coverage extending to reasoning-class models, the Model Context Protocol ecosystem, autonomous bug-bounty agents, and standardized agentic security benchmarks. Despite progress, challenges persist, such as model hallucination, context limitations, operating costs, and ethical concerns. Future directions include advancements in multi-agent collaboration, mechanisms for agent self-improvement, and improved transparency into agent behavior.

INDEX TERMS Large language models (LLMs), AI agents, cybersecurity, autonomous penetration testing, prompt engineering, vulnerability discovery, static application security testing (SAST), multi-agent systems, chain-of-thought (CoT), ReAct, Plan-then-Execute (PtE).

I. INTRODUCTION

A. CONTEXT AND MOTIVATION

Cybersecurity operations today are defined by an unprecedented scale and growing complexity. Enterprise networks, cloud infrastructures, and software supply chains evolve faster than conventional information security defensive measures can adapt. Classical approaches such as manual penetration testing, rule-based Static Application Security Testing (SAST), and procedural network scanning remain indispensable, yet they struggle to address the velocity and variety of modern threat landscapes [1], [2]. Prior

The associate editor coordinating the review of this manuscript and approving it for publication was Turgay Celik.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

work has explored automated penetration testing through other means, such as reinforcement learning for post-breach scenarios [3]. Recent advances in LLMs have introduced a new paradigm: intelligent agents capable of reasoning about context, interpreting code semantics, and orchestrating security toolchains autonomously. Research by Keltek et al. [1] on LLM-Supported SAST (LSAST) shows that coupling traditional static analyzers with LLM post-processors enables deeper semantic correlation across codebases, which is an ability unattainable in purely rule-driven systems. Likewise, Nakatani [4] and Luong et al. [2] demonstrate how multi-agent penetration testing frameworks can coordinate reconnaissance and exploitation through adaptive orchestration, integrating tools such as Nmap and dnsenum with reasoning modules that interpret intermediate findings. LLM-assisted fuzzing for software security represents another emerging application [5]. These innovations mark the emergence of AI-augmented security intelligence, systems that replicate aspects of expert judgment by analyzing results, prioritizing vulnerabilities, and refining actions in real time. Frameworks like VulnBot [6] and xOffense [2] exemplify this evolution, embedding LLM reasoning within feedback-driven pipelines that accelerate discovery and reduce human oversight. What distinguishes this technological moment is not merely faster automation but a deeper cognitive integration between reasoning and action. As explored throughout this paper, LLM-based agents represent an emerging class of cybersecurity systems capable of traversing the entire reasoning loop, from prompt to payload, bridging perception, interpretation, and exploitation in a unified and adaptive process.

B. PROBLEM DEFINITION AND GAP

Despite rapid progress in AI-augmented cybersecurity, current research remains fragmented across specialized domains. Individual studies have focused on narrow capabilities, such as autonomous web exploitation, code auditing, or network scanning, but few have examined how these components integrate into a unified, reasoning-driven workflow. Traditional surveys on AI in cybersecurity typically emphasize model performance and dataset composition, with less emphasis on agentic process design; that is, how perception, reasoning, and action are coordinated through modular LLM architectures. This fragmentation leaves several open gaps. First, the absence of a shared reference architecture obscures how tools like VulnBot [6], RapidPen [4], AutoPentester [7], or LSAST [1] operationalize the reasoning loop from observation to execution. Second, limited interpretability and lack of benchmarking make it difficult to assess reliability across frameworks. Third, existing literature seldom explains how prompting techniques, reasoning chains (ReAct, Plan-then-Execute), and structured workflows can be tuned for secure, verifiable automation.

Consequently, practitioners and learners in the field face a growing need for an integrative resource that encompasses these three aspects: 1) Explaining the underlying mechanisms of LLM-driven agents, 2) Synthesizing the diverse experimental systems into a coherent taxonomy, 3) Establishing conceptual continuity between traditional and AI-driven security pipelines. This paper aims to address these gaps by offering both a tutorial foundation and a comparative survey of intelligent security agents.

C. AIM AND SCOPE

Recent systems have demonstrated impressive capabilities in isolated tasks such as navigating web applications, interpreting code semantics, or coordinating external security tools; however, integrated guidance that explains how these components operate together within a coherent agentic workflow remains scarce. The following paragraphs clarify the scope of this survey and the audience it serves.

The scope of this survey focuses specifically on LLM-powered agents within cybersecurity, not on machine learning for cybersecurity as a whole. The intention is not to catalog every AI-based security tool. Instead, the paper examines how current agent architectures combine natural language reasoning, planning, memory mechanisms, and tool interaction to perform complex security operations with a degree of autonomy that was previously unattainable. By concentrating on these systems, the survey highlights the design patterns that shape their behavior, the operational loops that guide their decision-making, and the emerging conventions that define how they transform high-level objectives into actionable security outcomes.

Beyond exploring existing approaches, this survey also serves as an educational foundation. Readers who are new to agentic AI will find a structured introduction to concepts such as chain-of-thought reasoning, ReAct (Reason + Act) style prompting, function calling, and multi-step planning. They will benefit from comparative analysis across application domains, as well as a detailed walkthrough of the complete workflow that connects prompts to payloads. Through this dual purpose, both tutorial and survey, the paper provides a cohesive view of the mechanisms, capabilities, and limitations of current LLM-based security agents.

To sharpen differentiation from prior cybersecurity survey papers, this work centers on an operational lens: how LLM-based systems move from mission scoping to tool-mediated exploitation while preserving safety constraints, traceability, and verification checkpoints. The tutorial component is intentionally scoped to concepts directly needed for interpreting surveyed frameworks, while the comparative sections prioritize cross-framework synthesis and design trade-offs.

D. SURVEY METHODOLOGY

This survey follows a structured evidence-gathering process to improve transparency and reproducibility. The search process focused on peer-reviewed papers and high-quality preprints spanning approximately 2022 to early 2026, when modern agentic LLM security systems emerged rapidly. Query families included terms such as "LLM AI agents," "Plan-and-Execute reasoning," "tool use,"

"function calling," "penetration testing," "vulnerability scanning," "API penetration testing," and "self-improving agents."

Inclusion criteria required direct relevance to LLM-based agent architectures, workflows, or evaluations for vulnerability discovery, penetration testing, source-code security analysis, or network-security operations. Exclusion criteria removed studies centered only on non-agentic ML classifiers, purely theoretical discussions without architectural or operational detail, duplicate reports of the same system without substantial updates, and works lacking enough methodological detail to support reliable comparison.

Screening proceeded in three stages: (1) title/abstract filtering for domain fit, (2) full-text review for architectural and evaluation detail, and (3) subsection assignment by primary contribution (web/API testing, SAST/code auditing, network/infrastructure, or cross-cutting workflow/challenges). Because reported metrics are heterogeneous across papers, comparative tables retain source-reported values and explicitly mark missing fields as NR (Not Reported) instead of imputing estimates.

The resulting corpus primarily informed three application subsections in Section III, with additional foundational and safety/governance references used in Sections II, IV, and V. This process provides a reproducible, auditable survey backbone while preserving flexibility for a fast-evolving literature.

Following a PRISMA-style accounting (adapted for a literature survey), the search initially identified over 200 candidate records (based on deduplicated results across academic databases: IEEE Xplore, ACM Digital Library, Scopus, Google Scholar, and curated preprint sources including arXiv `cs.CR`, `cs.AI`, `cs.SE`). Title/abstract screening retained approximately 160 records; full-text review retained 155 unique sources that are cited in this manuscript (the count grew modestly from the initial submission due to 2025–2026 developments added during revision). The retained references distribute approximately as follows: about 18–22 inform the LLM and agentic-foundation tutorial in Section II, about 28–34 inform the three application subsections in Section III (web/API testing, SAST and code auditing, network/infrastructure scanning), about 16–20 underpin the prompt-to-payload workflow in Section IV, and about 30–40 ground the safety, governance, dual-use, and future-direction discussion in Section V. Counts are approximate because a substantial subset of references is cited in more than one section (e.g., MAPTA, AutoPT, HPTSA, IRIS, VulnBot, xOffense, and PentestGPT recur across the survey, workflow, and challenges sections). This survey's primary focus is offensive and evaluative agents for vulnerability discovery. Adjacent applications that share the same agentic substrate (malware reverse engineering, supply-chain package vetting as briefly discussed in Section III-B, defensive SOC automation, and LLM-based digital forensics) are acknowledged as promising extensions but are not surveyed comprehensively.

E. STRUCTURE OVERVIEW

After this introduction, Section II offers a tutorial-style explanation of the essential components that enable LLM-based agents to operate. It begins with the Large Language Model itself, including its handling of context, its instruction-following behavior, and its prompting strategies. The section then describes the agentic loop that links perception, reasoning, and action. It concludes by examining memory systems and tool-use interfaces, which together allow an agent to interact effectively with the external world.

Section III transitions from foundational concepts to practical applications by surveying how LLM agents are currently used across major security domains. These domains include web application and Application Programming Interface (API) penetration testing, automated source code auditing, and network and infrastructure scanning. Each subsection highlights the common techniques, reasoning strategies, and operational patterns observed across existing frameworks. The section ends with a comparative analysis that distills recurring design choices, limitations, and differences across the approaches reviewed.

Section IV provides a practical walkthrough of the full agent-driven security workflow, from mission scoping through payload refinement, demonstrating how high-level instructions become structured actions within an autonomous penetration testing lifecycle.

Section V addresses the open challenges involved in deploying or developing LLM-based agents. It discusses technical issues such as hallucination, context length constraints, and cost considerations. It then examines broader safety and ethical concerns, including containment, dual-use risk, and the potential misuse of autonomous capabilities. The section concludes by outlining future research directions, including advances in multi-agent collaboration and mechanisms that promote transparency and self-improvement.

Finally, the paper concludes by summarizing the central themes and emphasizing the significance of LLM-driven agents as a transformative step in automated cybersecurity operations.

II. ANATOMY OF AN LLM-BASED SECURITY AGENT

A. THE CORE ENGINE: THE LARGE LANGUAGE MODEL

1) CONTEXT WINDOWS

A central concept in understanding how LLMs process information is the context window, which refers to the amount of text that a model can process at once during inference. In essence, the context window functions as the model's working memory, enabling it to maintain coherence through sequential tokens and preserve dependencies in language generation [8]. Models such as GPT-4, Claude 3, and Llama 3 vary in their context window capabilities, with some extending to millions of tokens, as shown in Table 1. These extended windows allow LLMs to process longer sequences of text, which, in the context of this paper, are particularly useful for tasks in vulnerability discovery that involve analyzing extensive documentation, logs, or code.

TABLE 1. Representative LLM context window specifications (approximate vendor-reported values, 2024–2026 reasoning-class models; not benchmark results).

Model	Context Window	Output Limit	Avail./Cutoff‡
Gemini 1.5 Pro	∼1M	∼8K	Nov 2023
Claude 3.5 Sonnet	∼200K	4K–8K	Apr 2024
GPT-4 Turbo	∼128K	∼4K	Dec 2023
GPT-4o	∼128K	∼16K	Jun 2024
Llama 3.1 (405B)	∼128K	∼4K	Dec 2023
OpenAI o3	∼200K	∼100K	Apr 2025*
OpenAI o4-mini	∼200K	∼100K	Apr 2025*
DeepSeek-R1	∼128K	∼32K	Jan 2025*
Claude 3.7 / 4 Sonnet†	∼200K	∼64K	Feb–May 2025*
Claude 4.5 / 4.6 Opus	200K–1M	∼64K	Nov 2025–Feb 2026*
Gemini 2.5 / 3 Pro	∼1M	∼64K	Mar–Nov 2025*
Llama 4 (Maverick)	∼1M+	8K–32K§	Apr 2025*

‡Dates without * denote confirmed training-data cutoffs; dates with * denote model availability (knowledge cutoffs may be earlier).
†Claude 3.7 Sonnet and Claude 4 Sonnet are distinct releases grouped for brevity.
§Llama 4 output limit varies by deployment configuration.

The 2024–2026 reasoning-model class (e.g., OpenAI o3/o4-mini, DeepSeek-R1, Claude 3.7/4 Sonnet, Claude 4.5/4.6 Opus, Gemini 2.5/3 Pro, Llama 4) introduces a qualitative shift relevant to security agents: these systems allocate substantial *test-time compute* to internal deliberation chains before responding, going beyond reliance on parameter scale alone [9], [10], [11], [12], [13], [14], [15], [16]. For penetration-testing agents, this trades latency and per-call cost for stronger multi-step planning and self-verification, which can directly reduce hallucinated exploit hypotheses (Section V-A1) and improve plan quality in long-horizon agent loops. Several surveyed frameworks (e.g., AutoPT, HPTSA, MAPTA) report sensitivity to underlying model capability [17], [18], [19]; with reasoning models, planner roles benefit disproportionately while bulk tool-calling executors can often remain on cheaper, faster models. The cost discussion in Section V-A3 is updated to reflect this asymmetric deployment pattern.

However, empirical research shows that longer context does not always lead to better performance. Liu et al. [20] found that LLMs often exhibit a ''lost in the middle'' effect, where information located midway through long sequences is recalled less effectively than information at the beginning or end. Similarly, Levy et al. [21] demonstrated that model accuracy tends to plateau beyond a certain input length, indicating diminishing returns. These findings highlight that while extended context windows expand potential applications, they also introduce challenges such as performance degradation, higher computational costs, and Out-of-Distribution (OOD) effects related to positional encoding.

To address these problems, several strategies have been proposed. Summarization and truncation approaches help isolate salient information, thereby reducing sequence length without fully sacrificing context [22]. More recent architectural innovations such as SelfExtend introduce multi-level attention mechanisms that enable models to map unseen token positions into familiar ranges, effectively extending their usable context without additional fine-tuning [23].

Context windows represent both strengths and limitations for LLMs. They enable models to sustain reasoning over extended inputs, yet their fixed nature and susceptibility to performance decline with long text sequences require ongoing innovation. For LLM-based security agents, understanding and optimizing the context window is crucial, as the complexity of real-world vulnerability data often exceeds the reliable operating range of existing models.

2) PROMPTING TECHNIQUES

Prompting techniques are the practical means by which LLMs are ''programmed'' and directed toward specific tasks. In agentic systems, prompts define the procedures to follow, goals to pursue, and constraints to respect. Current publications lay out a toolkit that starts with basic zero-shot and few-shot prompts, where you either provide only the instruction or add a couple of labeled examples to shape the pattern the model should imitate [24]. Moreover, Chain-of-Thought (CoT) prompts request the model to demonstrate the steps, which increases multi-step reasoning, and self-consistency considers several reasoning paths and picks a majority answer to stabilize results [25], [26].

Another straightforward yet powerful method is role prompting whereby the model is informed to behave as an expert in a certain domain so that it forms its response in that format or tone [27]. For tasks that mix thinking and doing, prompts can incorporate reasoning with actions, like ReAct, which has the model explain what it is doing while it calls tools or looks things up. This reduces error cascades in tasks like question answering and web interactions [28]. Retrieval-augmented prompting inserts retrieved passages into the prompt so the model grounds its answers in those sources, improving open-domain question answering (QA) and reducing overall hallucinations [29]. For math, code, and structured problems, scratchpads and ''program of thoughts'' prompts help the writing of intermediate calculations or short programs before producing a final answer, thus enabling the model to self-verify [30], [31].

Verification-oriented prompts like Chain-of-Verification prompt the model to plan checks, answer them, and revise its output, which further reduces hallucinations. This is distinct from CoT, which produces step-by-step reasoning [32]. Finally, effective prompting also implies being cautious about the things you include. Controlled studies reveal that loading a large amount of irrelevant text or giving pathological instructions can confuse the reasoning process [33]. Table 2 summarizes these prompting techniques, their patterns, appropriate use cases, and associated trade-offs.

TABLE 2. Prompting techniques for LLM Agents: Patterns, Uses, and Trade-offs (conceptual summary of surveyed techniques, not empirical benchmark results).

Technique	Purpose	Pattern	When to use	Complexity	Benefits	Pitfalls	Cites
Zero-shot	Follow instructions with no examples	Plain task instruction with constraints and output format	Simple classification, drafting, quick turns	Low	Fast and minimal prompt engineering	Brittle on novel tasks	[24]
Few-shot	Shape behavior with a few labeled examples	Instruction plus 2–5 input output pairs	Patterned tasks where format matters	Low-Med	Stabilizes style and structure	Example bias; longer prompts	[24]
CoT	Improve multi-step reasoning	Ask for step-by-step thinking or show brief worked steps	Math, logic, planning tasks	Med	Higher accuracy on multi-step problems	Longer outputs; may overelaborate	[25], [26]
Self-consistency	Reduce variance in reasoning	Sample multiple CoT paths and pick majority answer	Hard reasoning with inconsistent outputs	Med	More stable answers than single CoT	More tokens and latency	[25], [26]
Role prompting	Set voice, scope, and stance	"Act as a … constraints: …"	Domain-specific tone and procedures	Low	Clear framing and adherence to norms	Overly rigid if mis-specified	[27]
ReAct (reason + act)	Mix reasoning with tool use	Alternate short reasoning with explicit tool calls	Tasks needing lookup or external tools	Med-High	Fewer error cascades across tool steps	Requires a runtime to execute actions	[28]
Retrieval-augmented prompting	Ground answers in sources	Insert top-k retrieved passages before the query	Open-domain QA and citation-heavy tasks	Med	Lower hallucinations; sourceable claims	Poor retrieval degrades answers	[29]
Scratchpad	Encourage intermediate work	Reserve a space for notes or calculations	Math, code, structured outputs	Med	Lets the model check its work	Can wander without clear stop signal	[30]
Program of Thoughts	Let the model write tiny programs	Ask for short code to compute or verify before answering	Numeric or algorithmic problems	Med-High	Executable checks reduce slip-ups	Needs execution or careful review	[31]
Chain-of-Verification	Plan checks, verify, then revise	List checks, answer them, then revise final output	High-stakes accuracy and safety	Med	Fewer hallucinations than plain CoT	More steps and longer prompts	[32]
Prompt hygiene	Avoid derailment and noise	Keep instructions concise, relevant, and testable	All tasks; especially long contexts	Low	More stable reasoning and outputs	Requires discipline selecting context	[33]

3) IMPORTANCE OF INSTRUCTION-FOLLOWING CAPABILITIES

One of the most appealing aspects of LLM-based AI agents is their assumed ability to follow instructions in accordance with the human-defined end goal. The ability of LLMs to follow instructions provided by human users is important for practicality and safety. LLM-based agents can be more helpful in real-world tasks if their instruction-following capacity is satisfactory [34]. Beyond this practical importance, problems can also arise from inadequate instruction-following. Research has shown that language models and thus the AI agents they empower can produce uncertain outputs that display bias or misinformation while also posing security and privacy risks involving malicious use or data leakage [35].

Different methods have been shown to better enable LLMs to follow instructions. Instruction-tuning is a method that provides LLMs with datasets that provide direction on determining instruction-output pairs that are desirable. These datasets can be divided into two categories: human-annotated datasets and LLM-synthetic datasets. Human-annotated datasets are defined by explicit human curation, while LLM-synthetic datasets are ones curated by other LLMs [34]. Another approach to achieve greater instruction-following capabilities is Transfer Learning which involves providing task-specific data to the model to enhance its performance related to specialized tasks. Finally, Alignment-tuning is a method that updates models by identifying undesirable responses and incorporating human feedback to reduce recurrence [35], [36].

Recent work shows that multiple factors can affect the instruction-following capability of LLMs like task familiarity, task difficulty, and prompt phrasing with the latter having a more adverse effect on the LLMs' ability to successfully follow given instructions. This has shown that Prompt Engineering (PE) can help increase the ability of

an LLM to follow provided instructions [37]. One possible advantage of using PE to improve instruction-following is that it does not require manipulation of the model's internal components, rather it targets the clarification of the input provided to the model [38].

The importance of instruction-following in LLMs presents many different challenges going forward as their use and integration into technologies like agentic AI systems becomes more predominant. Instruction-following improvement may come at a cost. Alignment-tuning helps LLMs follow instructions more safely, but this refinement comes with a decrease in model performance [34]. Furthermore, as research strives to enhance instruction-following in LLMs, such advancements may enable exploitation that produces harmful outputs leading to negative impacts [35].

B. THE AGENTIC LOOP: FROM THOUGHT TO ACTION

1) OBSERVE–THINK–ACT CYCLE

The Observe–Think–Act (OTA) cycle is the foundational loop of intelligent agents. At its core, it describes how an agent perceives its environment (observe), reasons over the information (think), and carries out a response (act). This structure, long established in robotics and classical AI, ensures that agents continuously update their internal state based on new inputs before deciding on the next step [39], [40]. Sapkota et al. [41] argue that OTA underpins both traditional AI agents, which rely on explicit sensing and planning modules, and emerging agentic AI systems, where large language models interpret textual prompts as observations, generate reasoning chains, and act through output or external tools in more open-ended domains.

Modern LLM-based agents such as GPT-4 and Claude 3 exemplify an OTA cycle. For instance, in Toolformer [42] the model examines an internal prompt (observation), plans whether to invoke an API (reasoning), and executes or refrains (action). In cybersecurity, an AI agent observes suspicious network logs, reasons by correlating them with known attack patterns, and acts by alerting analysts or automatically blocking traffic as demonstrated by VulnBot [6] and VulAgent [43].

Zhang et al. [44] note that traditional frameworks treat reasoning and acting as separate steps, with external prompts determining when to switch modes. Their proposed agent models internalize this loop by unifying CoT (reasoning) with Chain-of-Action (execution), effectively transforming the cycle into Thought–Action–Observation (TAO). This shift moves the agent from reactive to autonomous; note that TAO reorders the classical Observe–Think–Act (OTA) loop above for ReAct-style agents that interleave reasoning with tool use. Table 3 illustrates this cycle in a cybersecurity incident scenario, showing how an agent iteratively moves between thought (reasoning), action (execution), and observation (feedback). Each cycle shows the agent evaluating context, performing an action, and observing the outcome, which informs subsequent reasoning.

2) PROMINENT REASONING FRAMEWORKS

a: CHAIN-OF-THOUGHT

LLMs have immense capabilities in language understanding, but their ability to perform complex, multi-step reasoning remained limited until CoT prompting was introduced, a technique designed to elicit reasoning in LLMs by having them generate a sequence of intermediate steps that lead to a final answer [25]. CoT shows how reasoning itself can be demonstrated in context, guiding models to produce logically structured solutions through the few-shot and zero-shot learning frameworks [45]. Following its introduction, research that focused on deconstructing the mechanics behind CoT's success revealed that the critical factor is not necessarily the factual accuracy of each step, but rather the logical structure and sequential coherence of the demonstrated reasoning process itself [46]. The Counterfactual Chain-of-Thought (CCoT) approach leverages this principle to explain model decisions by generating alternative, hypothetical reasoning paths [47].

CoT prompting has evolved from a simple technique to boost performance into a fundamental concept for understanding and explaining LLM behavior. Below is an example of the Thought–Conclusion structure in a security context:

Prompt: ''You are a security analysis assistant. Explain your reasoning step by step. Identify whether the login pattern is suspicious.''

Model response with CoT:

- **Thought 1:** The logs show 19 failed login attempts from one IP in 12 seconds.
- **Thought 2:** The user account targeted is ''admin'', which is a high-privilege account.
- **Thought 3:** The geolocation of the IP has never appeared in this system before.
- **Conclusion:** This login pattern is highly suspicious and may indicate credential brute forcing.

b: ReAcT (REASON + ACT)

ReAct is a prompt-based framework that lets a language model mix its reasoning with concrete tool-use, so the agent can reason about what to do, take an action, read the result, then refine its plan in short cycles. Yao et al. [28] formalize this by expanding the agent's action space to include both environment actions and language thoughts and then alternating thought-action-observation steps during a trajectory. ReAct addresses common failure nodes of reasoning-only prompting. On HotpotQA and FEVER, ReAct mitigates hallucination and error by consulting an external Wikipedia API mid-reasoning, producing decision traces that humans can inspect [28].

The broader tool-learning literature frames this process as four linked stages: task planning, tool selection, tool calling, and response generation [48]. Integrating tools in this way improves robustness to input behavior and helps the model adapt to new environments, while also increasing user trust through traceable, evidence-based steps.

TABLE 3. Illustrative Thought–Action–Observation cycle in a cybersecurity incident scenario (pedagogical example, not empirical benchmark data).

Cycle	Thought	Action	Observation
1	"Network traffic from one host looks abnormal, possible data exfiltration."	Query SIEM and IDS logs for unusual outbound data transfers.	60 MB of encrypted outbound traffic detected from a finance workstation after business hours.
2	"Could this be legitimate backup traffic or an anomaly?"	Check scheduled backup jobs and correlate timestamps with observed traffic.	No authorized backups or sync process match the time window.
3	"This suggests a potential compromise. I should isolate the host."	Trigger endpoint isolation and capture memory image.	Host successfully quarantined. Forensic image stored in secure share.
4	"Now I need to confirm the root cause and prevent future recurrence."	Run malware analysis on memory dump and update firewall rules to block the detected domain.	Malicious PowerShell script identified, domain blacklisted, incident ticket closed.

For example, when an agent needs to check for open ports, it would follow this ReAct loop:

- **Thought:** I need to check for open ports. I should use the Nmap tool.
- **Action:** `nmap 203.0.113.17`
- **Observation:** Host is up. Open ports: 22, 80.
- **Thought:** I want to know what services are on those ports.
- **Action:** `nmap -sV -p 22,80 203.0.113.17`
- **Observation:** 22/ssh OpenSSH, 80/http nginx
- **Thought:** I have enough for the initial note. The target has SSH and a web server. I will record this and plan basic web checks next.

This example mirrors the ReAct loop: plan in language, select a tool, act, read the result, then update the plan [28].

c: PLAN-THEN-EXECUTE (PtE)

The Plan-then-Execute reasoning framework focuses on the distinction between the planning operation performed by an LLM and the subsequent execution of tasks [49]. A simple form of PtE improves upon few-shot CoT and zero-shot CoT techniques where the model is explicitly prompted to break the provided problem down into subtasks (the plan) and then carry out each part of its devised plan (the execution). This form, known as Plan-and-Solve (PS) prompting, becomes PS+ when additional task-specific reminders and constraints are injected into the prompt; in other words, the added detail is the distinguishing feature of PS+ relative to baseline PS [50]. AI agents can utilize a PtE-based architecture where a reasoning-intensive LLM acts as the *Planner* that divides the user-defined goal into subtasks, and an *Executor* that carries out each subtask in pursuit of the larger objective [49].

The following is an example of using PS+ prompting to guide an agent to utilize PtE to solve an information security focused problem:

Problem prompt: "I have a mail server I want to secure. I am using SMTP and IMAP for email services and must be able to use SSH to administer the server from a specific private IP address only. The only services that should be running are: SSH (TCP port/22), SMTP (TCP port/25), SMTP Submission (TCP port/587), IMAP (over TLS) (TCP port/993). I only have access to basic Linux networking tools (nmap, netstat, iptables, tcpdump). What actions should be taken to achieve this posture?"

Plan-then-Execute Elicitation prompt: "Let's first understand the problem, extract the relevant services and corresponding ports and devise a plan to achieve the desired security posture. Then let's carry out the plan step-by-step, ensure that initial steps (like scanning) are generated first before moving on. For each step (1) list the commands that should be entered, (2) the expected results, (3) and failover considerations."

The advantage afforded by the reasoning-intensive Planner comes with the trade-off of a longer "Time-to-First-Action" due to the increased upfront computation time. Planning is also vulnerable to hallucination which can result in illogical planning or failure to adhere to the planned tasks [51]. Several PtE-specific frameworks exist including LangChain, LangGraph, CrewAI, and Microsoft's AutoGen. HuggingGPT is an LLM-based agent that integrates PtE by using ChatGPT to break down user requests into smaller tasks and then selects specialized models to execute them [52]. Figure 1 illustrates the architectural interaction between the Planner and Executor agents in a typical PtE system. Step 1 represents the user-provided task or overall goal given to the agent. Step 2 represents the Planner agent breaking the overall goal into smaller subtasks to be completed downstream by the Executor. Step 3 represents the Executor agent observing the subtasks provided by the Planner to carry out in order to achieve the overall goal. Step 4 represents the Executor executing subtask(s). Step 5 represents the Executor providing updates to the Planner based on what subtasks it has completed, allowing the Planner to update its plan accordingly.

C. ESSENTIAL COMPONENTS

1) MEMORY

Memory is an essential part of what makes AI agents stay consistent and useful across different tasks, because it provides them with the ability to maintain context. For agents powered by LLMs, memory generally comes in two forms: short-term memory (context windows) and long-term memory (vector databases). A critical component of these models is their ability to maintain context, but they still

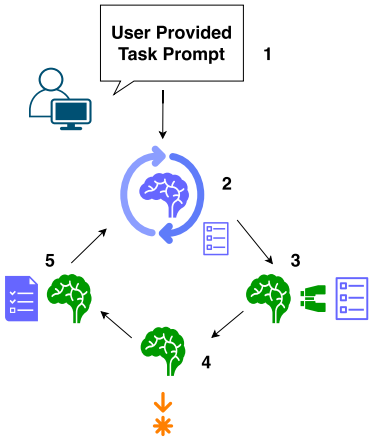


FIGURE 1. Conceptual Plan-then-Execute architectural diagram (illustrative, synthesized from surveyed literature) showing the interaction between Planner and Executor agents.

face challenges. These include being expensive to operate, struggling with complex decision making, and having trouble staying coherent during long conversations [53].

Short-term memory is like the model's temporary workspace, corresponding to the context window described in Section II-A1. It includes everything the model can currently see or consider and is limited by how much text it can handle at once; earlier parts of the interaction are forgotten once the sequence grows too long. Xiong et al. [54] show that this kind of short-term memory works well for quick reasoning, but the agent may lose track of the bigger picture during longer interactions unless there is a mechanism to retain older information.

Long-term memory lets the agent remember things beyond its short-term limits. This usually means saving valuable information in an external system, often called the vector database. Past interactions are converted into vector embeddings that represent meaning and are stored for later retrieval, so the agent can search and reuse relevant information without keeping raw text in context. Sun and Zeng [55] found that using a structured memory system helps the agent retrieve relevant information efficiently. Similarly, Qian et al. [56] introduced MemoRAG, a method that combines this type of long-term memory retrieval with the model's short-term reasoning, giving the agent both immediate coherence and the ability to recall useful information from the past. Inspired by human cognition, researchers classify long-term memory in LLMs into episodic memory (events), semantic memory (facts), and procedural memory (skills) [57]. Work on long-term memory in LLMs also applies decay and reinforcement strategies to prioritize frequently accessed information and manage growth over time [53].

2) TOOL USE ANDFUNCTION CALLING

LLMs move from text to action when they learn to produce structured function calls and when they are prompted with patterns that show when to use a tool, which tool to choose, and how to pass arguments. Two complementary strategies enable this behavior. Training-based methods teach the model to emit a consistent, machine-readable call format and to master skills such as function selection, argument binding, and multi-step chaining. Prompting-based methods keep the base model fixed and use short, program-style examples; a runtime executes the call and inserts the result back into the context [42], [58], [59], [60]. In both cases, a unified function schema and a clear list of available tools guide the model, so a request like "scan this host" or "run this code" becomes a concrete call such as `nmap_scan(ip_address)` or `run_code(python_script)`. For security agents, schema validation and verification (e.g., as in ToolACE) help ensure calls are well formed and that risky tools are gated when they affect real systems [60], [61].

Building on these foundations, Toolformer demonstrates that an LLM can teach itself tool timing and formatting by inserting calls into ordinary text only when they help prediction [42]. Scale and diversity matter for generalization and robustness. ToolBench constructs large datasets of real APIs, generates diverse instructions that require tool use, and annotates solution paths as chains of calls [58], [62]. ToolACE proposes adaptive task difficulty and dual-layer verification that catch name mismatches, missing required parameters, and hallucinated values [61].

Building on this foundation, function-calling LLMs extend agent capabilities well beyond Natural Language Processing tasks [63]. Several training- and prompting-based approaches have been proposed to instruct LLMs in tool use. Direct tuning of the model is one option [63]; a second tuning-based approach uses a model such as ChatGPT as a teacher to generate instruction-following data linked to tool use, as exemplified by GPT4Tools [64]. RestGPT targets RESTful APIs through a modular framework that facilitates API calling [65], and EASYTOOL addresses inconsistent or incomplete tool documentation [66]. MultiTool-CoT, in contrast, uses a concentrated prompting approach that bundles tool inventories and few-shot examples [67]. Because tool-equipped LLMs can produce real-world effects, erroneous tool returns can cascade into compounding errors; humans therefore remain responsible for outcomes, and such agents are best deployed as suggestion mechanisms under human oversight [63].

In late 2024 and 2025, two open standards have begun to consolidate this fragmented landscape into reusable interfaces. The *Model Context Protocol* (MCP), released by Anthropic in November 2024, provides a vendor-neutral protocol that lets agents discover and invoke tools, resources, and prompts exposed by external "MCP servers," similar in spirit to the Language Server Protocol but for agent-tool interaction [68]. MCP servers for filesystems, browsers, databases, and security tooling reduce per-framework integration cost and create a common substrate for sandboxing and audit. The *Agent2Agent* (A2A) protocol, introduced by Google

in 2025 and adopted by multiple vendors, complements MCP by standardizing inter-agent collaboration: capability discovery, task negotiation, and structured message exchange between agents that may run on different stacks [69]. For LLM-based security agents, MCP and A2A directly affect tool gating, least-privilege policy enforcement, and multi-agent coordination patterns examined later in Sections V-B and V-B4.

However, rapid MCP adoption (97 million monthly SDK downloads reported by early 2026) has outpaced security tooling [70]: the January 2026 "Clinejection" supply-chain incident compromised approximately 4,000 developer machines through a composed attack chain in which a malicious GitHub issue title induced a code assistant to run `npm install` on a poisoned package, exploiting trust boundaries between MCP-connected tools rather than compromise of a single server [70]. Public surveys from the same period reported that 41% of MCP registry servers accepted unauthenticated connections and that roughly 12% of audited agent skills exhibited malicious behavior [70]. In response, the MCP-38 threat taxonomy classifies 38 distinct attack categories specific to MCP ecosystems [71], the Open Web Application Security Project (OWASP) Agentic AI Top 10 (2026 edition) formalizes agent-specific risk categories [72], and Microsoft's MCP control-plane guidance for securing agent tool execution [73]. These developments underscore that the governance concerns in Section V-B6 are not hypothetical but already operational.

III. A SURVEY OF LLM AGENTS IN ACTION: APPLICATIONS AND TECHNIQUES

For convenience, Table 4 provides a brief context box for the principal LLM-based security frameworks discussed throughout this section. Acronyms and tool names are expanded on first use in subsequent prose.

TABLE 4. Framework Glossary: One-line Context for Surveyed LLM-based Security Systems (reference summary, not an evaluation).

Name	Brief description
AutoPT	FSM-based autonomous penetration testing agent [17]
HPTSA	Hierarchical Planning with Task-Specific Agents for web exploitation [18]
MAPTA	Multi-Agent Penetration Testing AI for web/API targets [19]
PentestGPT	Penetration Testing Task Tree (PTT)-driven LLM agent [74]
PentestAgent	Multi-phase agent scaffold with retrieval-augmented memory [75]
HackSynth	Containerized CTF-oriented LLM agent [76]
SqliGPT	Crawler–Replayer–Scanner pipeline for SQL injection [77]
IRIS	Whole-repository LLM-augmented SAST detector [78]
LSAST	LLM-Supported SAST hybrid auditing pattern [1]
RapidPen	Tool-orchestrated LLM pipeline for IP-to-shell recon [4]
VulnBot	Penetration Task Graph-driven multi-agent framework [6]
xOffense	Task Coordination Graph (TCG)-based multi-agent framework with fine-tuned models [2]
AutoPentester	LLM-orchestrated pentest framework with adaptive sequencing [7]
VulAgent	Multi-agent vulnerability validator with static-analysis verification [43]

A. WEB APPLICATION AND API PENETRATION TESTING

Penetration testing is an information security practice in which practitioners conduct sanctioned attacks against applications, networks, or systems in order to subvert the implemented security features [79]. By proactively identifying and managing risks before they are exploited, effective penetration testing can protect organizations from adverse financial and reputational consequences [80].

1) BACKGROUND OF WEB PenTest

LLM agents have evolved from proof of concept into end-to-end web and API penetration tests that find and validate flaws. Recent systems target classic web flaws such as SQL injection, cross-site scripting, and broken access control while operating inside safety-scoped sandboxes. In controlled evaluations modeled on the Natas wargame, teams report challenges that explicitly include Cross-Site Scripting (XSS), SQL injection, directory traversal, authentication quizzes, and cookie analysis [76]. Multi-agent frameworks extend this idea to the open web by splitting work across specialized actors such as navigator, recon, and exploitation [19].

2) RECENT ADVANCES OF AI AGENTS FOR WEB PenTest

Literature on agent capability is converging on three patterns. First, containerized testbeds keep agents safe while giving them realistic tools. HackSynth wraps an agent in Docker with strict firewall rules and a reproducible target set [76]. Second, multi-agent plans improve coverage. Multi-Agent Penetration Testing AI (MAPTA), a multi-agent framework whose web-focused agents pass intermediate artifacts such as Uniform Resource Locators (URLs), response bodies, and suspected payloads, extends agentic coverage across the open web [19]. Third, purpose-built systems such as PentestAgent define a scaffold of reconnaissance, enumeration, exploitation, and reporting [75].

A fourth pattern has emerged rapidly in 2024–2025: *computer-use* and *browser-use* agent substrates. Anthropic's Computer Use API (October 2024) gives Claude direct keyboard, mouse, and screenshot-level control of a virtual desktop [81]; OpenAI's Operator (January 2025) provides a browser-resident agent capable of navigating, filling forms, and chaining tool calls in a sandboxed environment [82]; and open-source projects such as Browser-Use [83] and Skyvern [84] expose programmable web-browsing agents that combine LLM reasoning with vision-grounded DOM/UI interaction. For LLM-based web and API penetration testing, these substrates substantially lower the engineering cost of (i) realistic interaction with single-page applications

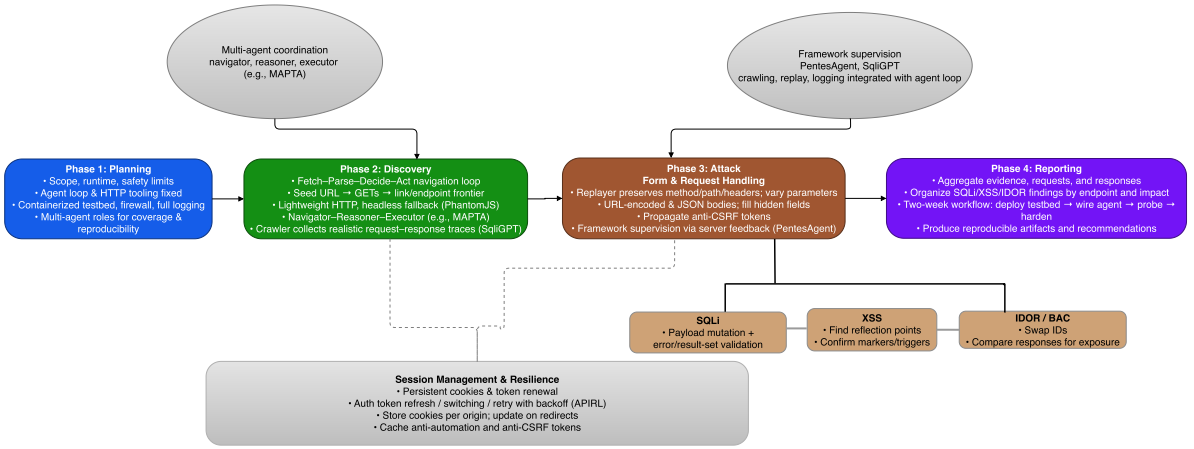


FIGURE 2. Conceptual four-phase workflow (illustrative, synthesized from surveyed frameworks) for LLM-based web and API penetration testing: planning, discovery, attack, and reporting.

and authentication flows, (ii) replay and auditing through screenshot/event traces, and (iii) reproducible sandboxing of exploration sessions. They also expand the agent-specific attack surface (Section V-B): screenshots and rendered page content become first-class observations, which makes indirect prompt injection through page text or alt-text a concrete threat that web-pentest agents must explicitly model. More broadly, natively multimodal models such as Gemini 3 Pro [15] open the door to vision-grounded vulnerability detection (for instance, identifying visual phishing artifacts, recognizing misrendered UI states indicative of DOM manipulation, or interpreting CAPTCHA and 2FA flows), extending the agent's perceptual repertoire beyond text-only DOM parsing. Figure 2 depicts the four-phase workflow commonly adopted by LLM-based web and API penetration testing systems.

SQL injection remains a central target. SqliGPT validates a workflow with three modules: a crawler that collects request–response pairs, a replayer that can mutate inputs and resend them, and a scanner that evaluates database-error signatures [77]. The prompt-to-SQL (P2SQL) literature warns that models embedded inside web apps may actually generate the exploit. A recent study documents prompt-to-SQL injection attacks across popular frameworks [85].

3) PHASES FOR LLM-BASED WEB AND API PENETRATION TESTING

The four-phase workflow referenced in Figure 2 can be summarized as follows. Phase 1 (Planning): Planning sets scope, runtime, and safety limits and fixes the agent loop and HTTP tooling. Containerized testbeds with strict firewalling and complete logging provide a controlled environment for end-to-end runs and align with multi-agent designs that coordinate roles for coverage and reproducibility [19], [75], [76]. Phase 2 (Discovery): Agents follow a cycle of fetch, parse, decide, and act: from a seed URL they issue requests, follow links or discover endpoints, and maintain a frontier of targets. Effective steps are often short HTTP calls plus simple text processing; headless runtimes (e.g., for JavaScript-heavy pages) serve when static parsing is insufficient [76]. In multi-agent settings, a navigator proposes next hops and a reasoning agent scores their utility before the executor makes the request, which helps on multi-step tasks such as staged XSS [19]. SqliGPT's crawler supports discovery by collecting request–response pairs and preserving cookies, headers, and encodings for later replay [77]. Phase 3 (Attack): Form filling and payload injection require accurate request extraction and replay. A common pattern is to build or reuse a minimal representation of the original request, preserve method and headers, and vary only parameters of interest; agents often try URL-encoded and JSON bodies, propagate anti-CSRF tokens, and escalate from benign probes to riskier payloads based on server feedback [75], [76], [77]. Session management is critical: persistent cookies and token renewal must be handled explicitly; API-focused work (e.g., APIRL (deep reinforcement learning for REST API fuzzing), a learning-based REST API fuzzing framework) integrates auth refresh, throttling, and retry into the agent's policy [19], [86]. For SQL injection, agents combine payload mutation with server-side error or result-set validation; for XSS, they search for reflection points then confirm execution via markers in returned HTML or stored XSS; for insecure direct object reference (IDOR) and related broken access control, they swap identifiers in URLs or bodies and compare responses [19], [75], [77]. Phase 4 (Reporting): Reporting aggregates evidence, actions, and outcomes into reproducible artifacts. The literature points to a practical baseline: a safe testbed, robust HTTP tooling, cookie and token persistence, a disciplined replay loop for forms, and guards that can veto risky outputs [75].

4) REPRESENTATIVE FRAMEWORKS

a: AutoPT (FSM-BASED)

AutoPT is a framework inspired by the Finite State Machine (FSM) model. This design helps address memory-related problems, agents getting stuck on tasks, and inference limits

that can lead to overall failure, in contrast to attack-tree-style designs such as PentestGPT's penetration testing task tree (PTT) [17], [74]. The FSM-inspired Penetration Testing State Machine (PSM) has two core states: the Agent state and the Rule state [17]. The Agent state comprises LLM agents and performs via different substates covering scanning, reconnaissance, and exploitation; it consumes a prompt with operational description, role-playing instructions, tool information, and a thought-action response format, then iterates tool use until a human-defined limit or an exit decision. The Rule state interprets the Agent's output via predefined rules, selects vulnerabilities for exploitation by severity and ease, and checks success against expected outcomes. Parsing functions between states extract only the information relevant for tool calling or vulnerability categorization. AutoPT's toolset is organized into Terminal (a Kali Linux Docker container with root access), Playwright (headless browser interaction via a LangChain-based library), and Search (e.g., first-page Google results for reconnaissance) [17]. Compared to the ReAct framework, AutoPT successfully completed double the number of simple tasks and nearly 10 times more complex ones; its modularization distributes information across Agent states and helps reduce the impact of hallucinations and simple-task failures. AutoPT shows economic promise, averaging $0.02423 per website test (GPT-4o mini model usage). The framework was tested against a practical target (HackTheBox UnderPass machine [87]) and successfully compromised the system 80.6% faster than the fastest human [17]. Figure 3 illustrates the AutoPT FSM workflow.

Notably, AutoPT demonstrates strong results with open-source models such as DeepSeek and Qwen, which performed close to or even surpassed the commercial GPT-4o model. This success with open-source models indicates practicality regarding local model deployments that could enable offline testing and address privacy concerns. Even with the improvements provided by AutoPT's architecture, hallucinations during longer and more complex reasoning chains can degrade outcomes. Model security policies can also be an encumbrance, sometimes refusing to carry out penetration testing tasks even when provided role-playing prompts [17].

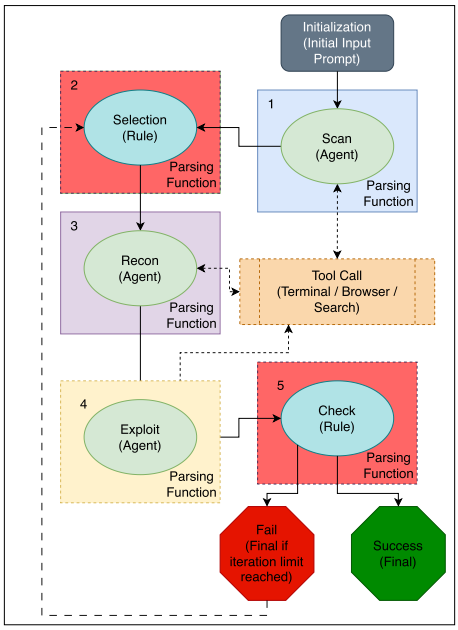


FIGURE 3. Conceptual AutoPT FSM workflow (illustrative, synthesized from source descriptions) showing state transitions between scanning, reconnaissance, and exploitation phases.

b: HPTSA (HIERARCHICAL PLANNING AND TASK-SPECIFIC AGENTS)

HPTSA stands for "Hierarchical Planning with Task-Specific Agents." HPTSA is a framework that focuses on teams of AI agents collaborating to complete penetration tests. The framework architecture is composed of three major agent types: a hierarchical planning agent, a management agent for the task agents, and task-specific expert agents. The hierarchical planning agent instructs the manager agent based on exploration actions it performs on the target environment. The manager agent determines which task-specific agents to employ and can provide previous information produced by agents if further testing needs to be conducted. The task-specific agents perform specific exploits delineated by six categories: XSS, SQL injection (SQLi), Cross-Site Request Forgery (CSRF), Server-Side Template Injection (SSTI), the Zed Attack Proxy (ZAP), and a generic agent. The agents are enabled to perform penetration testing activities through access to external tools, documentation (on the order of 5–6 documents per vulnerability category), and a prompt template customized per vulnerability with relevant exploitation information [18]. The agents are orchestrated using LangGraph's agent graph with LangChain; to reduce token usage, client-side HTML is stripped of extraneous tags before being passed to agents, since HTML often dominates token counts [18]. Figure 4 shows the HPTSA interaction architecture.

HPTSA's toolset includes Playwright for website access, a terminal, tools for file management, and both OWASP ZAP and sqlmap for the agents in those respective categories. In evaluation against a simpler agentic framework that gives GPT-4 external tools and Common Vulnerabilities and Exposures (CVE) descriptions, HPTSA achieved successful exploitation at 1.8 times the success rate of that baseline (with CVE info) given multiple attempts, and up to two times better when the baseline lacked CVE information [18]. GPT-4 outperformed open-source models within HPTSA, which were unable to exploit vulnerabilities in the reported tests. The identified cost of utilizing HPTSA was $4.39 per run for the exploit-capable GPT-4 powered model. HPTSA showcases the ability to integrate information from different

task-specific agents to fulfill the current overall testing objective, and its modularity allows task-specific agents to reduce the need to return to previous steps. Limitations include elevated refusal rates and faulty testing strategies with open-source models, and difficulty for exploitation agents when no obvious attack path is present [18].

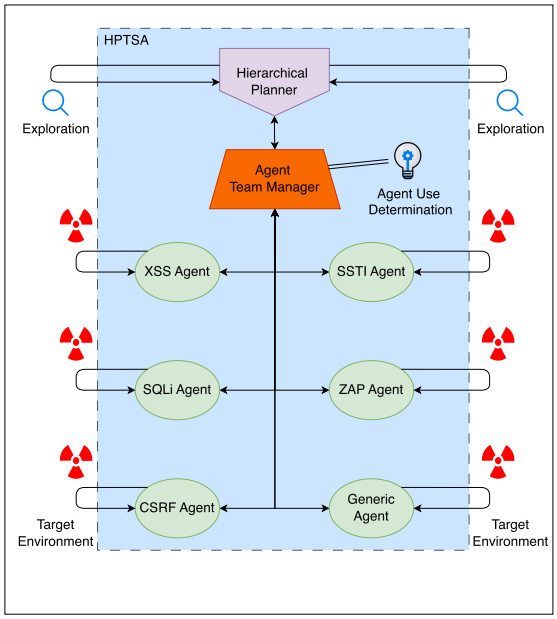


FIGURE 4. Conceptual HPTSA interaction architecture (illustrative, synthesized from the source description) showing hierarchical planning agent, management agent, and task-specific expert agents.

B. AUTOMATED SOURCE CODE AUDITING (SAST)

1) BACKGROUND OF TRADITIONAL AUTOMATED SOURCE CODE AUDITING

When testing an application or program's source code for weaknesses and vulnerabilities, two complementary approaches are typically employed: dynamic analysis, which observes a program's behavior during runtime, and static analysis, which inspects the source code before execution. This subsection focuses on the latter. SAST is a cybersecurity method that automates static analysis to detect vulnerabilities early in the Software Development Life Cycle (SDLC) by scanning for insecure patterns and rule violations in the source, byte, or assembly code [1]. Transformer-based and embedding-based methods have been applied to vulnerability discovery in lifted code [88], [89]. SAST analyzes source code for potential vulnerabilities to SQL injections, buffer overflows, cross-site scripting, and others. Since SAST operates prior to compilation and execution, it helps developers address security weaknesses and logic errors in the earliest stages of development, reducing both time and cost associated with post-deployment fixes [90].

SAST workflow begins by parsing the source code into a machine-readable format. The tool applies rule-based pattern matching or regular expressions to detect known coding flaws or deviations from secure code standards. The detected issues are grouped into vulnerability classes based on predefined rule sets tied to the programming language, configuration standards, and security checkpoints. The process usually involves several layers of analysis: semantic, data flow, control flow, and structural analysis to identify syntax errors, insecure configurations, and unsafe logic pathways. Once the vulnerability is detected, SAST tools generate reports indicating the location of the issue, making it easier for developers to pinpoint and resolve it, as shown in Figure 5 [90].

Traditional SAST tools offer advantages such as enabling early vulnerability detection, working across the entire codebase without requiring execution, and integrating easily into development workflows so that developers can identify weaknesses even as they write code. However, they also have limitations such as being difficult to configure initially, being language-dependent and prone to false positives. Sometimes projects even require layered SAST tools to achieve sufficient coverage. They also lack the ability to reason contextually on developer intent or logic [90].

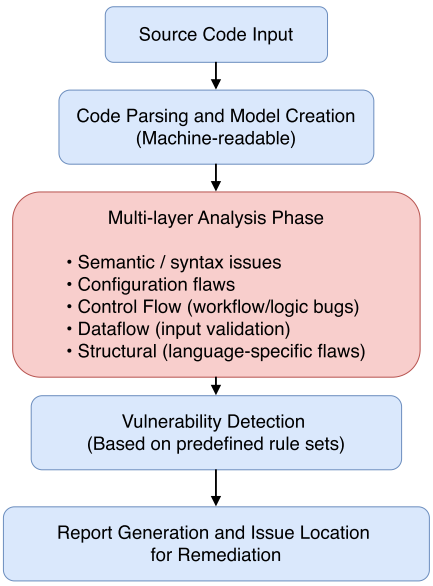


FIGURE 5. Conceptual workflow of Static Application Security Testing (illustrative summary of typical SAST stages, not a benchmark of any specific tool).

Popular SAST tools include Fortify, SonarQube, CodeQL, and Semgrep. Each relies heavily on syntactic and rule-based matching, enforcing coding standards and flagging insecure patterns in source code. CodeQL performs semantic queries to detect unsafe data flows and logic flaws, as demonstrated in large-scale studies analyzing hundreds of open-source projects [91]. Similarly, Semgrep and SonarQube apply customizable, rule-driven analysis to ensure code quality and security compliance, with comparative evaluations demonstrating their effectiveness and limitations in static analysis workflows [92]. Traditional SAST methods are rule-based and highly dependent on predefined coding standards, making them strong but rigid. As modern software grows in

complexity, purely syntactic methods are unable to interpret context or intent within code. As discussed in Section II, LLMs are capable of semantic reasoning, context awareness, and adaptive learning, making them suitable for integration into the SAST workflow.

2) RECENT ADVANCES OF AI AGENTS FOR SAST

LSAST introduces a new class of hybrid auditing frameworks that amalgamate traditional SAST techniques with the contextual reasoning capabilities of LLMs. In a typical LSAST architecture, the SAST engine performs an initial scan using established static analysis rules to flag syntactic and structural anomalies; the resulting potential vulnerabilities and code snippets are then passed to an integrated LLM for deeper semantic reasoning and correlation across the codebase [1]. According to Keltek et al. [1], LLMs, while effective at identifying syntactic flaws, are limited by their static training data. To overcome this limitation, the authors propose incorporating Retrieval-Augmented Generation (RAG), which provides dynamic, internal knowledge sources so the model can access up-to-date information such as recent CVEs while avoiding direct, uncontrolled external API calls and mitigating privacy and data-leakage risks [1], [93]. Frameworks such as Secure Multifaceted-RAG (SecMulti-RAG) extend this by integrating internal enterprise documents, curated expert content, and in some cases filtered external LLM outputs, using a locally hosted model for most tasks and reserving external LLM access for vetted inputs [93]. Compared to traditional SAST, LSAST systems that integrate RAG can better infer developer intent, interpret code in semantic context, and prioritize findings by exploitability over raw rule-trigger frequency, leading to fewer false positives [1].

LSAST agents extend traditional static analysis along two main dimensions. First, contextual reasoning: through integrated retrieval and reasoning loops, they correlate issues across function boundaries and application-level context, uncovering higher-order vulnerabilities that often evade purely rule-based methods [1], [78]. Second, adaptive learning: whereas traditional tools require manual updates to rule sets when new Common Weakness Enumerations (CWEs) arise, LSAST frameworks use RAG and incremental fine-tuning to incorporate emerging vulnerability knowledge, and dynamic retrieval allows internal knowledge bases to be updated for better recall on zero-day or recently published CWE categories [1], [93].

A notable example is the IRIS framework proposed by Li et al. [78], which integrates LLMs with static analysis to enable whole repository reasoning and uses LLMs to infer taint specifications and analyze context, reducing reliance on manually crafted rules. When evaluated on real-world Java projects, IRIS powered by GPT-4 detected 55 vulnerabilities, significantly more than traditional tools like CodeQL, which detected only 27, and identified several previously unknown vulnerabilities that demonstrate the potential of LLMs to uncover subtle, context-dependent flaws missed by conventional analyzers [78]. Transformer-based embeddings have also been applied to vulnerability detection in lifted (binary) code, complementing source-level LSAST [94]. LLMs are also being applied to intelligent fuzzing for software security [5]. Representative LLM-assisted fuzzing systems include ChatAFL, which uses an LLM to learn input-format grammars and generate protocol-aware mutations on top of AFL-style coverage feedback [95]; Fuzz4All, a universal fuzzer that prompts an LLM to synthesize and refine inputs across multiple target languages and compilers [96]; and Google's OSS-Fuzz LLM augmentation effort, which uses LLMs to generate or improve fuzz harnesses for open-source projects, scaling coverage that previously required manual harness authoring [97]. These systems share a common pattern with the agentic SAST tools surveyed above: LLMs supply semantic context and generative variation that conventional analyzers lack, while deterministic tooling (AFL/libFuzzer, sanitizers, coverage feedback) provides ground truth that filters hallucinated outputs.

Two recent high-profile demonstrations underscore the trajectory of agentic source-code auditing in 2024–2025. First, Google's Project Zero "Big Sleep" agent (an evolution of the earlier "Naptime" framework) reported the first known LLM-agent discovery of an exploitable vulnerability in real-world production code: an exploitable stack buffer underflow in the SQLite library, found before the affected build reached an official release [98]. Big Sleep combines an LLM agent with structured tools (code reading, debugger, fuzz harness, and reasoning scratchpad) and is positioned as a complement to coverage-guided fuzzing, not a wholesale replacement. Second, the U.S. Defense Advanced Research Projects Agency's *AI Cyber Challenge* (DARPA AIxCC) concluded its finals at DEF CON 33 in August 2025 [99], [100]. Team Atlanta won the competition; across all seven finalist cyber-reasoning systems, competitors collectively identified 54 of 63 synthetic vulnerability challenges (86%) and patched 43 (68%) while analyzing 54 million lines of code, and additionally uncovered 18 previously unknown real-world vulnerabilities; all finalist CRS codebases were released as open source. These results provide strong public evidence that LLM-augmented program analysis can complement, and in some cases extend, the reach of conventional SAST at non-trivial scale.

Beyond government challenges, autonomous bug-bounty agents have moved from prototype toward operational deployment. XBOW reports that its agents submitted over 1,060 vulnerabilities on HackerOne and reached the top of HackerOne's global leaderboard (vendor-reported figures, independently unverified as of early 2026) [101]. The same vendor reports matching a principal human pentester's 40-hour assessment in 28 minutes on vendor-designed benchmarks [101]. In March 2026, XBOW was credited with three critical Microsoft RCE vulnerabilities (CVE-2026-21536,

CVE-2026-32194, CVE-2026-32191) [102]. In March 2026, AWS launched its Security Agent for on-demand penetration testing as a generally available service across six regions, offering autonomous 24/7 multi-cloud testing that compresses typical engagement timelines from weeks to hours [103].

A related application of agentic SAST is software supply-chain security, where LLM agents analyze open-source packages for malicious behavior. CHASE (Collaborative Hierarchical Agents for Security Exploration) deploys a multi-agent Plan-and-Execute architecture with deterministic security tools to dissect PyPI packages, achieving 98.4% recall with only 0.08% false-positive rate on a corpus of 3,000 packages [104]. IntelGuard uses RAG over curated threat-intelligence reports, reaching 99% accuracy on 4,027 real-world packages and discovering 54 previously unreported malicious packages when deployed in a live monitoring configuration [105]. These systems share the same architectural patterns as the penetration-testing agents surveyed above (LLM reasoning + deterministic tool ground truth), but finer-grained indicator-level reasoning remains more challenging than binary malicious/benign classification [104], [105], indicating that verification layers remain essential.

Figure 6 illustrates the evolution from traditional SAST architectures to modern LSAST systems that incorporate RAG for enhanced vulnerability detection.

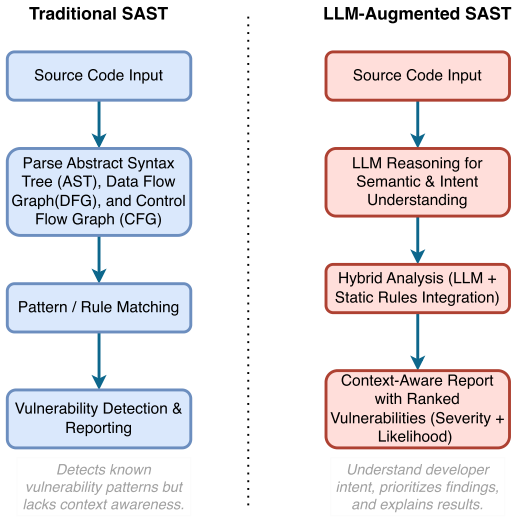


FIGURE 6. Conceptual evolution of Static Application Security Testing architectures (illustrative comparison) from traditional SAST to LSAST with RAG.

3) CHALLENGES AND RESEARCH DIRECTIONS FOR LSAST

LSAST raises several interrelated challenges. False positive mitigation remains a core concern, since neural and LLM-based detectors often favor recall over precision and can overwhelm developers with alerts; improving evaluation methodology and realistic benchmarks is key to reducing misleading results; data snooping in evaluation can inflate reported performance of ML-based detectors and must be addressed [1], [78], [106]. Dataset and benchmark design for ML-based security analysis, including graph-based representations for classification tasks, has received attention [107]. Explainability and trust require that LLM-generated findings be accompanied by verifiable reasoning, such as data-flow traces, counterexamples, or symbolic checks, so that engineers can validate and reproduce results [78]. Security and privacy risks arise when proprietary source code is processed by remote LLM APIs, including potential data exfiltration, memorization, or leakage; this motivates hybrid deployment patterns such as on-premise or locally hosted models, encrypted inference, and strict data governance, as exemplified by SecMulti-RAG and similar designs [93], [108]. Scalability and performance are also at issue: larger models incur higher compute, latency, and cost, complicating integration into continuous integration pipelines. Promising research directions include lightweight or distilled models, selective or triggered analysis that invokes deep LLM analysis only when certain conditions are met, and on-premise deployment to balance capability with cost and privacy [1].

C. NETWORK AND INFRASTRUCTURE SCANNING

1) BACKGROUND

Network and infrastructure scanning forms the frontline of external security assessment. It identifies live hosts, open ports, exposed services, and misconfigured network assets that together constitute an organization's attack surface. Common tools include Nmap for host, port, and service enumeration and DNS scanners for record and zone analysis [109]. Emerging automation frameworks such as RapidPen and xOffense integrate multi-tool orchestration within agentic pipelines to perform comprehensive reconnaissance autonomously [2], [4]. RapidPen is an LLM-driven pipeline that automates IP-to-shell probing by sequencing reconnaissance tools; xOffense is a multi-agent framework that fine-tunes Qwen to orchestrate scanning, reasoning, and exploitation tasks [2], [4]. Traditional workflows are procedural and human-driven: an operator executes sequential scans, reviews results, and then plans subsequent actions. Automating this process with AI-driven agents aims to accelerate reconnaissance, reduce manual oversight, and enable automated reasoning over sequencing, prioritization, and exploitation paths. RapidPen has demonstrated completion in 200-400 seconds per run at modest cost of $0.30–$0.60 per test, illustrating substantial gains in speed and efficiency; such frameworks reduce manual workload and enable scalable, repeatable assessment across large network environments [2], [4]. Table 5 contrasts traditional human-driven steps with their agentic counterparts.

2) AGENT ARCHITECTURES FOR ORCHESTRATING RECON TOOLS

In the context of AI-driven penetration testing, orchestration refers to the automated coordination and management of

TABLE 5. Conceptual comparison of traditional human-driven versus AI agentic workflows for network and infrastructure scanning (illustrative, synthesized from surveyed frameworks).

Human-Driven Workflow	AI Agentic Workflow
Manually launch scan	Planner agent schedules scan
Manual result parsing	Analysis agent parses scan output
Manual target prioritization	Reasoning agent prioritizes targets
Manual exploit launching	Exploit agent executes controlled tests
Manual result review	Reasoning agent evaluates and logs results
Manual next-step planning	Planner agent selects and sequences next action

interdependent security tasks (reconnaissance, scanning, exploitation, and reporting) through an intelligent control layer. This top layer governs the overall workflow, ensuring that each stage runs in the correct sequence and under appropriate contextual constraints; it is typically realized as a cognitive planning module or LLM-based strategy agent that determines which tests to perform, in what order, and according to what environmental conditions or observed outputs. By continuously evaluating intermediate results and aligning them with defined objectives, the orchestration layer maintains a logical and adaptive flow across all phases of testing [6], [7], [75]. Frameworks such as PentestAgent, VulnBot, and AutoPentester exemplify this LLM-driven orchestration; RapidPen and Assumed-Breach PenTester integrate feedback loops that dynamically adapt subsequent actions based on observed outcomes [4], [110].

Integration combines multiple tools (command-line scanners, parsers, and vulnerability analyzers) into a unified, interoperable agentic workflow. Recent frameworks embed traditional utilities such as Nmap and dnsenum within structured agent pipelines, exchanging results in standardized formats; each agent specializes in one testing phase and communicates findings through structured task objects that encode results, confidence scores, and context, enhancing cross-agent collaboration and adaptive sequencing [2], [6], [75]. The execution layer is the operational counterpart to orchestration: lightweight, task-specific agents receive commands from a central planner, run tools or deliver payloads, and return structured outputs to higher-level reasoning modules, preserving traceability and auditability across runs. VulnBot employs a task-graph architecture to manage dependencies between phases and coordinate execution across multiple agents; AutoPentester emphasizes LLM-driven orchestration to dynamically prioritize and sequence actions [6], [7]. Benchmarking of autonomous penetration testing agents relies on standardized datasets, task dependencies, and metrics to assess orchestration quality, decision-making accuracy, and system robustness [111].

3) CORE CAPABILITIES ENABLED BY AGENTIC RECON

Figure 7 presents the layered architecture of an AI agentic penetration testing system, showing how planner, execution, and reasoning components interact during network assessment. Agentic penetration testing frameworks improve adaptability, analytical depth, and operational transparency across reconnaissance and exploitation. As illustrated in Figure 8, planner agents determine scan type, scope, and timing; execution agents coordinate tool operations and collect results; and the reasoning agent analyzes outputs to produce a structured vulnerability report. Four capabilities characterize these systems. First, adaptive scan selection and prioritization lets agents adjust scanning strategies in real time based on prior results and environmental context (e.g., speed, stealth, or coverage). Frameworks such as PentestAgent, VulnBot, and RapidPen implement adaptive loops that modify scan intensity, timing, and target selection from prior observations and workflow dependencies, replacing static pipelines with responsive orchestration [4], [6], [75]. Second, cross-source correlation and reasoning synthesizes outputs from multiple reconnaissance and scanning tools into a unified view of the network; xOffense and AutoPentester use LLM-based reasoning modules to infer probable software versions and CVE exposures, reducing manual cross-referencing and providing a coherent view of the attack surface [2], [7]. Third, recursive automation and feedback allows agents to chain actions and trigger follow-up scans or enumeration from intermediate findings. RapidPen and VulnBot implement this so that results from one phase inform the scheduling and selection of follow-up tasks, improving coverage and vulnerability discovery [4], [6]. Fourth, human oversight and explainability keeps operators in control of safety-critical decisions through audit trails and interpretable rationales; PentestAgent, AutoPentester, and xOffense support decision histories, human-readable rationales, and operator review or intervention on potentially disruptive actions, ensuring transparency and auditability [2], [7], [75].

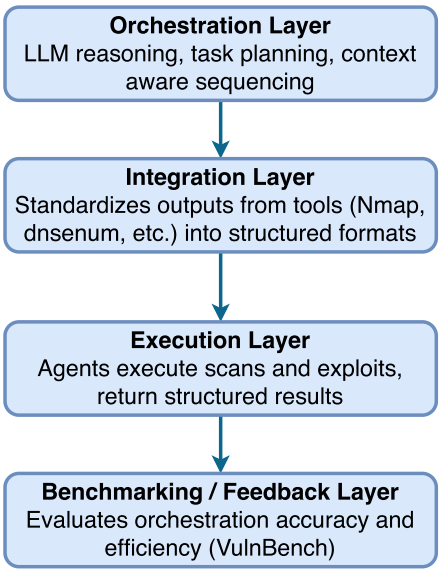


FIGURE 7. Conceptual layered architecture (illustrative reference model, synthesized from surveyed frameworks) of an AI agentic penetration testing system.

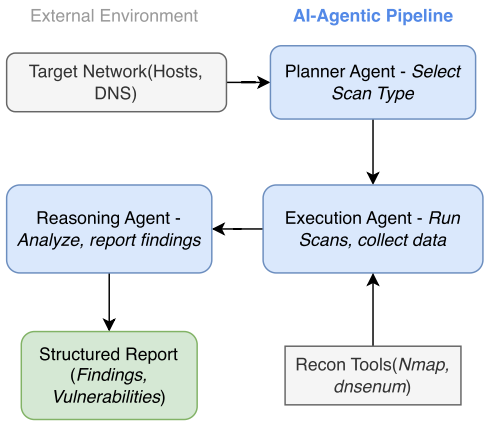


FIGURE 8. Conceptual orchestration diagram (illustrative, synthesized from surveyed frameworks) of agentic network scanning, showing interaction between an LLM-based agent, reconnaissance tools (Nmap, dnsenum), and the target network environment.

4) AGENT REASONING ABOUT NETWORK TOPOLOGIES

Enterprise networks can be modeled as graphs of hosts, users, services and their interactions; nodes represent entities (machines, accounts, services) and edges represent relationships (authentication, communication, trust). Once an attacker gains initial access, they use lateral movement to reach deeper into the network, which is a phase reflected in frameworks such as the MITRE Adversarial Tactics, Techniques, and Common Knowledge framework (MITRE ATT&CK) and the Cyber Kill Chain [112]. Lateral movement can be treated as a planning problem: the attacker must plan a sequence of hops from a compromised system to a target node. Modeling the network as a graph allows agents to reason about feasible paths and weak points [113]. Bowman et al. modeled authentication logs from enterprise networks as a graph of entity-authentications and used this for lateral movement detection; such modeling helps both defenders and testers identify at-risk systems and evaluate how controls or segmentation affect reachability [114].

Three families of approaches support this reasoning. First, symbolic planners and path-search agents use a formal model of the network (nodes, edges, known vulnerabilities, access controls) and apply search or planning algorithms to find feasible attack paths; reasoning is interpretable and can encode constraints (e.g., excluding certain credentials), but full path enumeration becomes costly as network size and uncertainty grow [112], [113]. Second, graph-based machine learning treats the network graph as input to a learning system (e.g., a graph neural network) that learns latent representations to predict attacker movements or anomalous transitions; Bowman et al. demonstrate an unsupervised graph-learning pipeline for detecting anomalous authentication events indicative of lateral movement. This approach scales better to large, dynamic networks and can capture subtle patterns, at the cost of lower interpretability [114]. Third, LLM-based reasoning agents combine structured data (network graphs, system logs) with text from security documentation or incident reports to identify attack paths and defenses; they can process cybersecurity knowledge bases, topologies, and Open-Source Intelligence (OSINT) to generate detailed attack plans [115]. These systems move beyond pure analysis toward autonomous decision-making about network operations, but their reasoning and outputs still face challenges in accuracy and verifiability [112].

Defenders rarely have full knowledge of every asset or exploit chain, so agents must reason under uncertainty; evaluation focuses on path correctness and operational usefulness (e.g., whether outputs help prioritize mitigations). Bowman et al. report 85% true-positive and 0.9% false-positive rates when applying their graph-based method to enterprise authentication data [114]. Topology-reasoning agents can reveal hidden privilege-escalation chains and high-impact controls (e.g., segmentation of key accounts), and graph-based detection can reduce false positives compared with rule-based heuristics; however, they depend heavily on data quality and completeness, and the same techniques raise dual-use and ethical concerns. Smiliotopoulos et al. emphasize defensive use, human-in-the-loop workflows, and explainability; hybrid models (symbolic plus learning) are often preferred to balance transparency with scalability [112], [114].

D. COMPARATIVE ANALYSIS

Table 6 summarizes the key characteristics of representative LLM-based security frameworks surveyed in this section. The comparison reveals several patterns. First, containerized execution environments (Docker, Kali) are nearly universal, reflecting the security community's emphasis on controlled testing. Second, multi-agent architectures appear predominantly in web penetration testing, where task decomposition (reconnaissance, exploitation, reporting) maps naturally to specialized agents. Third, the choice of reasoning framework varies by task complexity: simpler scanning tasks favor ReAct-style loops, while comprehensive penetration tests benefit from Plan-then-Execute architectures that separate strategic planning from tactical execution. Finally, cost efficiency varies significantly, with lighter models (GPT-4o mini) enabling economical large-scale testing while more capable models (GPT-4) achieve higher success rates on complex targets. To separate empirically evaluated systems from conceptual architectures and to increase methodological clarity, Tables 7 and 8 provide additional synthesis with explicit NR markers for unavailable source-reported fields.

In parallel with the framework-specific evaluations summarized in Table 6, the community has converged on a small set of standardized benchmarks that allow head-to-head comparison of LLM-based security agents across studies. Cybench packages 40 professional CTF tasks from four competitions with subtask difficulty annotations and execution-time profiling, providing a calibrated proving ground for end-to-end agent capability and risk [116]. NYU CTF Bench offers a scalable open-source corpus of CTF challenges

spanning categories such as web, reverse engineering, cryptography, and pwn, designed specifically to evaluate LLM agents in offensive security [117]. AutoPenBench focuses on penetration-testing scenarios in containerized network environments and explicitly distinguishes in-vitro from real-world tasks [118]. InterCode-CTF provides interactive sandboxed environments well suited to ReAct-style agents [119]. Meta's CYBERSECEVAL cyber-security evaluation series (most recently CYBERSECEVAL 3) extends these capability evaluations with explicit risk axes (insecure-code generation, cyberattack assistance, and prompt-injection susceptibility), giving deployment teams a structured way to compare model and agent risk profiles [120]. Finally, AgentDojo focuses on the evaluation of agent-specific attack surface, particularly indirect prompt injection through tool returns and untrusted retrieved content [121], complementing the offensive-capability benchmarks above.

Adoption of these benchmarks alongside framework-specific reporting would substantially improve cross-study comparability in this rapidly evolving area. A 2026 systematic evaluation of 10 frontier models across all 200 NYU CTF Bench challenges reports Claude 4.5 Opus achieving a 59% solve rate and Gemini 3 Pro reaching 52%, with environment configuration (a custom Kali Linux image with 100+ pre-installed tools) contributing a +9.5 percentage-point improvement over a standard Ubuntu baseline, which underscores that tooling access is at least as important as raw model capability [122]. Newer agent architectures explicitly address failure modes on harder tasks: PentestGPT v2 introduces difficulty-aware planning and achieves up to 91% task completion with 39–49% relative improvement over baseline agents on the XBOW benchmark [123]; AWE combines vulnerability-specific analysis pipelines with LLM orchestration for 87% XSS and 67% blind SQLi success on the same benchmark suite [124]; and CTFExplorer provides an open-environment benchmark with 40 vulnerable web services that moves evaluation beyond closed-world binary CTF scoring [125]. Nevertheless, a significant gap persists between benchmark performance and real-world effectiveness: CTF challenges offer well-defined, single-solution tasks, whereas production environments involve complex authorization boundaries, rate limiting, and multi-step exploit chains with ambiguous success criteria. Vendor-reported operational data from autonomous agents such as XBOW (HackerOne submissions and credited production CVEs) illustrates both promise and limits: reported successes on vendor-designed benchmarks do not guarantee comparable performance on hardened production targets [101], [102].

IV. THE WORKFLOW: FROM INITIAL PROMPT TO FINAL PAYLOAD

Section II introduced the foundational components of LLM-based agents, and Section III surveyed their applications across security domains. This section synthesizes these elements into a cohesive operational workflow, demonstrating how an autonomous penetration testing engagement progresses from initial mission definition to successful exploitation. The workflow presented here draws on patterns observed across the surveyed frameworks and distills them into four generalizable phases.

Four steps are essential when using agentic AI for penetration testing, as illustrated in Figure 9. Step 1 focuses on presenting the agent(s) with the penetration testing task. Step 2 involves the agent(s)' reasoning for environment exploration and tool use. Step 3 involves hypothesizing possible vulnerabilities and verifying them. Step 4 involves payload generation and refinement.

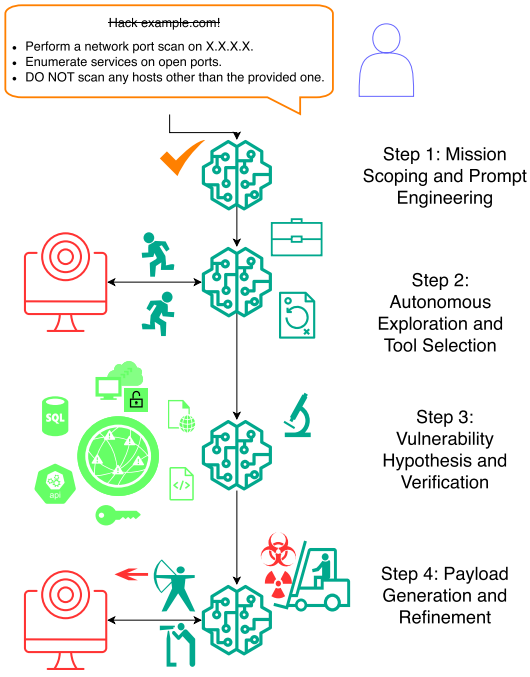


FIGURE 9. Conceptual agentic penetration testing workflow (illustrative, distilled from surveyed frameworks): from prompt to payload in four steps.

A. STEP 1: MISSION SCOPING AND PROMPT ENGINEERING

Before an automated tester sends a single request, it needs clear limits and plain instructions. Mission scoping and prompt engineering both give it. Scoping identifies the systems in play, allowed time windows, and permitted actions. Prompt engineering turns that scope into steps the agent can follow. Human teams already do this with rules of engagement that say what to touch, what to avoid, and how to handle risky actions [79], [126]. The National Institute of Standards and Technology (NIST) AI Risk Management Framework (AI RMF) outlines the same idea for AI systems: document intended use, map risks, and manage behavior over the lifecycle [126]. For an LLM agent, these controls live in the system prompt and related policy. Vague instructions

TABLE 6. Comparison of representative LLM-based security frameworks (source-reported metrics; heterogeneous evaluation setups).

Framework	Domain	Architecture	Reasoning	Environment	Performance / Cost
AutoPT [17]	Web PenTest	FSM-based	PSM (Agent + Rule states)	Docker/Kali	\$0.024/test; 80.6% faster than human baseline
HPTSA [18]	Web PenTest	Multi-agent hierarchy	Hierarchical planning	Sandboxed	\$4.39/run; 1.8× success rate vs. baselines
SqliGPT [77]	SQL Injection	Modular pipeline	Crawler-Replayer-Scanner	Controlled testbed	Automated detection with reduced false positives
IRIS [78]	SAST	LLM + Static analysis	Whole-repo reasoning	Offline analysis	55 vulns detected (vs. 27 for CodeQL alone)
RapidPen [4]	Network Recon	Multi-tool orchestration	Feedback-driven	Docker	200–400s/run; \$0.30–0.60/test
VulnBot [6]	Network PenTest	Task graph	Penetration Task Graph	Containerized	Structured multi-phase coordination
xOffense [2]	Network PenTest	Multi-agent	Task Coordination Graph	Docker	Fine-tuned Qwen competitive with GPT-4

Only frameworks with sufficient empirical evaluation detail are included; see Table 4 for the full glossary of surveyed systems.

TABLE 7. Evaluated Systems vs. Conceptual/Architectural Frameworks.

Framework	Type	Primary Domain	Evidence Basis	Notes
AutoPT [17]	Empirically evaluated	Web PenTest	Reported runtime and cost comparisons on controlled targets	Strongly operationalized FSM pipeline
HPTSA [18]	Empirically evaluated	Web PenTest	Reported success-rate and run-cost comparisons	Hierarchical multi-agent orchestration
RapidPen [4]	Empirically evaluated	Network Recon	Reported per-run time and cost ranges	Tool-orchestrated reconnaissance workflow
IRIS [78]	Empirically evaluated	SAST	Reported vulnerability-detection deltas vs baseline tooling	Hybrid LLM + static-analysis validation pipeline
VulnBot [6]	Architecture-forward with partial evaluation	Network PenTest	Task-graph architecture and staged workflow evidence	Comparative metrics reported as NR in several dimensions
xOffense [2]	Architecture-forward with targeted evaluation	Network PenTest	Task Coordination Graph plus model-performance observations	Cross-model comparisons reported for selected settings

TABLE 8. Evaluation context and metrics transparency (NR = Not Reported).

Framework	Targets / Dataset Context	Trials	Human Assist	Time Metric	Cost Metric	Reported Statistical Detail
AutoPT [17]	CVE-style and CTF-like targets	20 (reported)	Low after setup (reported workflow)	Yes	Yes	Limited variance reporting; confidence intervals NR
HPTSA [18]	Selected CVE targets (manually exploitable set)	14 target set (reported)	Setup/prompting required; runtime autonomy emphasized	Partial	Yes	Significance tests NR
RapidPen [4]	Containerized lab recon scenarios	NR	NR	Yes	Yes	Variance/confidence NR
IRIS [78]	Source-code auditing dataset with CodeQL baseline	NR	Validation stages documented	NR	NR	Confidence intervals NR
xOffense [2]	Network pentest task graphs and benchmarked scenarios	NR	NR	Partial	NR	Significance reporting NR

Note: values are transcribed from source papers and are not normalized across heterogeneous targets, environments, and scoring protocols. NR denotes fields not explicitly reported in the cited work.

invite drift; specific instructions anchor behavior and make results auditable.

First, write the goal in plain terms. Avoid writing "test security." Write what, where, and how. Name assets in scope, name the vulnerability classes that matter, and state limits on the test. NIST SP 800-115 treats this as table stakes: list targets, allowed techniques, timing, data-handling rules, rate limits, and reporting paths [79]. Rules of engagement then spell out how testing happens in practice, including "no production state change," finding validation, and who to call if something breaks. For an automated agent, encode all of this in the system or policy prompt so the role, boundaries, and output format are explicit.

Second, make instructions you can verify. ''Hack this website'' cannot be enforced. A specific task can. For example: (1) Test the target site and look only for reflected and stored XSS on specific pages. (2) Work in read-only mode: do not submit any form that saves data. (3) Limit traffic to 25 requests per minute. (4) For each issue, return a JSON object with the field's `url`, `param`, `payload`, and `evidence_snippet`. (5) Write the report in two parts titled ''Findings'' and ''Method'' and keep the ''Method'' section under 200 words. These constraints narrow actions and create outputs a script can check. Instruction-Following Evaluation (IFEval) formalizes ''verifiable instructions'' like ''answer in JSON'' or ''write 450–500 words,'' and scores compliance with simple structure and string checks [127]. Instruction Hierarchy Evaluation (IHEval) extends this to system-level rule conflicts [128]. When scope, safety limits, rate caps, and formats are expressed as verifiable instructions, compliance can be graded before a human reads the report [127], [128].

Third, put safety rules first. In an agent, the ''prompt'' is layered. A system prompt sets roles and strict rules. User prompts request actions. Safety and policy should be placed at the system level so that later requests cannot override them. Recent evaluation work separates system-level rules from user requests and checks whether models honor the system rules under conflict. Jailbreak and prompt-injection surveys show adversarial input can still make models ignore guardrails or leak restricted content; today's defenses reduce but do not eliminate that risk [129]. Clear system-level constraints, reinforced by examples and checked automatically, are a practical way to keep an agent within bounds [126], [128], [129].

Fourth, specify the evidence you want to collect. Do not ask the agent to ''report what you found.'' Tell it what evidence to collect and how it must look. For example, require each candidate finding as a JSON object {`url`, `param`, `payload`, `evidence_snippet`} plus trimmed HTTP excerpts. Tight format and length rules create a reliable handoff to triage code or a human reviewer. IFEval shows why this works: when outputs have checkable properties like ''these keys exist'' or ''exactly two sections,'' compliance can be measured at scale with deterministic checks [127]. SP 800-115 stresses planning the report format, logging, and data-handling rules up front so findings are reproducible and clear [79].

Fifth, treat prompts as an optimization loop. Prompting is not a magic phrase. Surveys show techniques like few-shot, CoT, and self-consistency are dials, not silver bullets; results depend on wording, task framing, and how success is measured [130]. DSPy treats prompts as declarative modules you can tune end-to-end to improve instruction-following [131]. A practical loop looks like this: define a testable output contract; run the agent; measure contract compliance; adjust examples, constraints, or wording; repeat until behavior is consistent.

Sixth, match prompts to the agent's loop. Write tasks to match how the agent actually works. Systems like PentestGPT track state, break work into subtasks, and coordinate tools for penetration testing [74]. They benefit from prompts that already decompose work into checkable subtasks and evidence items, not a single open-ended goal. Keeping safety boundaries, scope, and evidence rules at the system level, and using user-level prompts only for ''what to try next,'' fits this pattern and shows gains in controlled studies.

In short, concise scope beats vague asks. Open-ended instructions like ''hack this website'' do not name assets, limit impact, or define how to judge results. A scoped instruction that specifies target, safety limits, and output format does; it aligns with rules of engagement and AI governance and lets instruction-following checks grade whether the agent stayed within policy [79], [126], [127], [128]. That verifiable control is the baseline for safe, repeatable testing.

B. STEP 2: AUTONOMOUS EXPLORATION AND TOOL SELECTION

1) BEFORE EXPLORING AND TOOL USE

Once the agent has appropriately scoped the mission and objectives for a penetration test, the planning phase ends and discovery begins. The discovery phase requires interaction with the environment to obtain information and determine its current state. Standard information to acquire from the target can be host IP addresses, system information like hostnames and accessible file shares, and application service information [79]. The agent must be able to reason and select tools to obtain the data needed for this phase and advance the test. Once the agent has received its testing objective, such as an IP address, URL, IP/port number combination, or a natural language prompt, it will begin the necessary interaction with the target environment. The ability of the agent to reason about which tools to use based on its observations and act on this reasoning supports exploration and a successful discovery phase that builds a foundation for later phases.

2) REASONING FOR EXPLORATION AND TOOL USE

Research shows that there are multiple approaches to elicit the reasoning for exploration and tool selection required for penetration testing activities. These various approaches show that the location of these capabilities within various agentic penetration testing frameworks are diverse [2], [17], [18], [19], [74], [76], [77].

PentestGPT implements a dedicated Reasoning Module into its design. This module deduces the overall testing objective into a structure resembling an attack tree (PTT). This becomes a critical point of reference for further reasoning and exploration of the environment. The module then reviews and judges the viability of possible sub-tasks from the tree that can advance the test. The sub-task with the highest chance of success is provided to the next module

that formulates the appropriate tool call such as an nmap command for a service enumeration sub-task [74].

A similar approach can be observed in use by the xOffense framework. Within xOffense, a Task Orchestrator creates a Task Coordination Graph (TCG). Each node in the TCG has multiple attributes, including a directive that describes an activity like network discovery. Another important attribute that helps with reasoning about tool use is the operation attribute which informs the command generating part of the framework on whether the current task will require commands or direct user intervention [2]. Using its custom FSM architecture, AutoPT performs reasoning and invokes tools for penetration testing activities like exploration and follow-on exploitation. Upon initialization, the system immediately utilizes a scanning agent to gather information on the provided target that is then routed to the next state in the FSM [17].

HackSynth presents a more straightforward approach with its implementation. The system makes use of a Planner module to observe the current state of the target system while also considering previous actions to determine the next command to issue. This is driven by system prompts that the module ingests instructing it to frame the activity as a Capture The Flag (CTF) exercise [76]. The MAPTA framework showcases agentic reasoning for exploration and tool usage that depends on its two different operating modes. Depending on the mode that MAPTA is deployed in: CTF (single-agent) or Real-world (multi-agent) the reasoning may occur in a pentest reasoning purposed agent or in a multi-tasking agent [19].

3) SIMILARITIES IN APPROACHES OBSERVED

Literature on developed frameworks reveals emergent patterns that enable them to reason and respond to observations during the testing process. Organization of observations made can provide paths forward for reasoning and next-step actions. Some implementations have dedicated graph structures that the agentic systems create, and update based on their penetration testing activities [2], [74], [132]. This approach helps the agentic system to break down broader penetration testing goals into sub-tasks that can then be used as a basis for executing tool calls for action. A broader goal like "perform reconnaissance on target" would be decomposed into "scan for open ports" then "determine active applications and services." These sub-tasks would represent atomic actions that can be taken to advance the test with direct tool calls [74]. Existing information security frameworks like MITRE ATT&CK have been used to create these structures [132].

Another important observation is the judgment function of agentic penetration testing systems that determine which paths to pursue for exploration and later tasks. Focusing the efforts of agents and highlighting the most favorable path forward plays a key role in reasoning during the test and the possible outcomes. A favored approach to achieve this capability involves the agentic system making judgments about the viability of testing paths generated from its observations. The part of the framework responsible for reasoning prioritizes the observations and follow-on actions linked to them if it is determined that they provide a higher probability of testing success [74], [76], [77].

Agentic penetration testers must invoke the correct tools for initial observations and follow-on testing actions based on those observations. Prompting is a popular method used in numerous ways to imbue agents with the knowledge of the correct tools to call for their desired actions. Prompting agents to think about what tools to use is a direct approach [74]. Directing agents to be more specific about tool selection and calls by asking them to select a single terminal command with the greatest likelihood of success based on their observations has also been implemented [76]. A more detailed approach involves providing prompts to agents that describe the tools available to them and examples of the tools being used to drive reasoning and further action [17].

4) KEEPING CONTEXT MOVING FORWARD

Penetration testing agents that explore environments and prepare for further exploration, vulnerability analysis, and exploitation need to effectively keep track of what they learn. A significant consideration in maintaining the context of the test information is the restricted context sizes of agents [17]. Memory management is needed to help agents remember previous insights and actions. Agents' context windows are analogous to working memory, while external databases can be used to provide an implementation for long-term memory [57]. Some agentic penetration testing frameworks employ this approach. Using RAG, agents can store information that is acquired from exploration in an external database, ensuring it is not lost due to context limitations. The information in this database can be used in later phases for vulnerability exploitation [75].

Summarizing insights gathered from exploration and tool outputs has been applied to help maintain context in agentic penetration testing frameworks [2], [75], [76], [132]. Prompting has been used to encourage agents to summarize the key findings of their observations and the output of tools they invoke. This summarized information can then be used to determine the next steps to take that best advance the test [132]. Combining summarization through prompting and storing the summaries in external databases has also been used to maintain reconnaissance information persistence [75]. Specialized approaches like using custom memory optimization frameworks show that relevant information can be kept while disregarding extraneous content. This results in improved efficiency and the ability to manage long context workflows [2], [133].

C. STEP 3: VULNERABILITY HYPOTHESIS AND VERIFICATION

1) HYPOTHESIS FORMATION

Autonomous security testing agents form initial hypotheses about the types of vulnerabilities that exist and which attack

paths are worth exploring based on observations gathered during reconnaissance. At this step, the agent synthesizes reconnaissance observations such as unusual error codes, latency irregularities, repeated parameter reflections or system fingerprints into vulnerability hypotheses without defaulting to brute-force payload generation. As demonstrated by Nakano et al. [132], an effective strategy is to anchor the LLM's reasoning in a deterministic task tree constructed from the MITRE ATT&CK framework. Their guided reasoning pipeline constrains the agent's choice of Tactics, Techniques, and Procedures (TTPs), the standard hierarchy used in MITRE ATT&CK to describe adversary behavior at strategic, technical, and operational levels. For example, limiting exploration to well-defined tactics and techniques drawn from MITRE ATT&CK like input validation, injection-style vulnerabilities, or authentication weaknesses can suppress hallucinated or operationally meaningless actions. By mapping observed system signals like an HTTP 500 response on input submission, distinctive latency spikes, or reflected user input appearing in output to nodes in this task tree, the Planner can produce plausible and actionable hypotheses within the structure of known adversarial behaviors.

In their evaluation, Nakano et al. [132] applied this guided reasoning approach to Llama-3-8B, Gemini-1.5, and GPT-4 across ten HackTheBox scenarios consisting of 103 subtasks. The agent achieved subtask completion rates of 71.8%, 72.8%, and 78.6%, respectively; compared to only 13.5% (Llama-3-8B) and 16.5% (Gemini-1.5) for self-guided baselines, which also consumed significantly more model queries. Even GPT-4 benefited, rising from 75.7% to 78.6%. These results indicate that grounding hypothesis formation in a structured threat model sharply reduces exploratory waste, improves task alignment, and mirrors the workflow of experienced human penetration testers: observe system behavior, map it to known tactics, and formulate disciplined, interpretable candidate attack paths.

2) HYPOTHESIS VALIDATION AND EXPERIMENT DESIGN

Once the agent formulates a candidate hypothesis, such as the possibility that an input parameter might trigger an injection flaw or bypass an authorization boundary, it must shift from conceptual reasoning to targeted empirical testing. The ATT&CK-aligned task-tree constraint introduced by Nakano et al. [132] (Section IV-C1) generalizes to validation: each experimental probe should map to a standardized technique within the same task graph, ensuring continuity between the reasoning that generated the hypothesis and the procedures used to test it.

This structure appears more explicitly in multi-agent penetration testing systems such as VulnBot [6]. VulnBot organizes reconnaissance, scanning, and exploitation through a Penetration Task Graph, enabling agents to select probes that follow defined state transitions and avoid unconstrained generative exploration. Validation actions, such as probing an endpoint, manipulating a parameter, or checking a suspected behavior, are thus grounded in graph-defined semantics, reducing arbitrary experimentation and improving interpretability.

A complementary validation-oriented design is presented in VulAgent [43]. VulAgent embeds hypothesis trigger path planning within a multi-agent pipeline, where specialized agents construct potential trigger paths and a validation module verifies their feasibility using static analysis tools. This approach substantially reduces false positives by ensuring that proposed vulnerabilities correspond to real execution flows and satisfiable triggering conditions.

Drawing from these architectures, the validation flow in an autonomous penetration testing agent typically follows a structured sequence: (1) select a procedure from a structured attack tree or task graph; (2) generate a minimally invasive experiment or payload aligned with that procedure; (3) execute it under controlled conditions via execution agents; (4) capture and normalize outcomes, such as error traces, reflection behavior, or authentication anomalies, for subsequent reasoning. By following this disciplined flow, the system avoids ad-hoc testing and produces interpretable, audit-friendly results aligned with real-world penetration testing methodology.

3) VERIFICATION AND ADAPTIVE REASONING

After an experiment is executed, the reasoning agent must evaluate the system's response in a structured way that remains faithful to the original hypothesis. As in hypothesis formation (Section IV-C1), verification should be grounded in the same deterministic ATT&CK-aligned task tree [132]; constraining interpretation within a predefined set of tactics and techniques reduces hallucinations and prevents the agent from overfitting on superficial text patterns. Verification thus becomes a question of semantic alignment: does the observed behavior correspond to the technique the agent believed it was testing?

In VulnBot [6], the outputs of each probe are fed back into the Penetration Task Graph, allowing the system to interpret the results in terms of progression along a structured pentest workflow. Although VulnBot does not include a standalone verification module, its design treats successful or failed probes as state transitions, such as advancing, stalling, or redirecting the pentest. This ties confirmation or reflection of hypotheses to the workflow-level semantics encoded in the graph, not to isolated textual cues alone.

VulAgent [43] provides an explicit model for hypothesis verification through its multi-agent static analysis pipeline. Here, validation agents assess whether the hypothesized trigger path is actually feasible by checking control flow dependencies, required preconditions, and contextual constraints within the code. When mismatches arise, the system reduces the hypothesis confidence and attempts structured refinements, including adjusting payload assumptions, modifying the triggered sequence, or constructing an alternative hypothesis path that better fits the observed constraints.

Across these systems, a consistent adaptive reasoning pattern emerges: (1) compare observed behavior against structured expectations from the attack tree or task graph; (2) update hypothesis confidence using an evidence-based revision process; (3) refine the hypothesis; (4) escalate or abandon based on accumulated evidence and workflow progression. This loop mirrors scientific inquiry (hypothesize, test, observe, revise) while maintaining procedural and semantic alignment with established adversarial tactics.

D. STEP 4: PAYLOAD GENERATION AND REFINEMENT

Payload generation in autonomous penetration testing is a structured process that transforms validated reconnaissance data into actionable exploit attempts. Existing systems follow the same progression: gather evidence, generate candidate payloads, test them, update the internal state, and refine or discard attempts based on results. This can be seen across SqliGPT, MAPTA, xOffense, and AutoPT, which all connect payload generation directly to tool output and system feedback, not to static payload lists [2], [17], [19], [77].

1) DATA TO CANDIDATE PAYLOADS

Payload generation begins only after the agent has collected enough validated context to justify a hypothesis about a vulnerability. SqliGPT performs this transition by analyzing its crawler output inside a Strategy Selection module that ranks likely injection points, then produces targeted SQL injection payloads customized to each parameter [77]. MAPTA evaluates hypotheses in its Coordinator Agent, which identifies plausible attack vectors and dispatches probing actions providing the substrate for initial payloads [19]. xOffense uses structured metadata in its Task Coordination Graph to guide exploitation attempts; nodes embed directives and operational attributes that help determine both the attack goal and the type of payload to produce [2]. AutoPT's finite-state architecture performs the same process by routing reconnaissance results from its scanning agent into the Selection state, which generates and ranks initial exploitation paths and their corresponding payloads [17]. Across these systems, payloads originate from structured reasoning elements such as ranked parameters, hypotheses, or graph nodes, not from ad-hoc prompting alone.

2) TESTING, FEEDBACK, AND STATE UPDATES

After generating an initial payload, frameworks rely on direct system feedback to decide whether the attempt should proceed, repeat, or be abandoned. SqliGPT validates payload impact by observing SQL error responses or behavioral deviations, then passes these results to a parameter generation stage that adapts subsequent sqlmap usage [77]. MAPTA cycles between hypothesis generation, probing, and observation; its Coordinator Agent reevaluates hypotheses after each tool output to confirm or reject the suspected vulnerability [19]. xOffense enforces this loop by attaching viability and operational rules to each TCG node, giving the framework a structured way to evaluate whether an attempted action remains sound or requires adjustment [2]. AutoPT embeds this evaluation directly into its state machine: when a payload fails, the system discards the attempt, re-evaluates the available exploitation vectors, and automatically backtracks or generates a refined payload [17]. In all cases, exploitation is treated as a measurement-driven loop, not a single-shot attempt.

3) REFINEMENT AND ITERATION

Successful exploitation in these systems depends on structured refinement. Instead of producing one payload per vulnerability hypothesis, the agent generates multiple candidates and iteratively adjusts them using prior output. SqliGPT demonstrates this by progressively adjusting SQLi payloads as it learns which parameters respond to error-based, boolean-based, or time-based probes [77]. MAPTA similarly refines its probing strategy as its Coordinator gathers evidence, pruning hypotheses that lack support and strengthening those that receive confirming feedback [19]. xOffense uses node attributes in the TCG to prevent repetitive or invalid command generation, enabling cleaner iteration toward a viable payload [2]. AutoPT's refinement process is explicitly documented: the system generates an initial payload, observes the result, removes failed paths, retrieves previous state information from its database, and crafts a new payload that incorporates what it has already learned [17].

AutoPT provides a concrete example of refinement on a real target. In a case study using the HackTheBox "UnderPass" machine [87], AutoPT moved through a full refinement cycle while acting on live system responses. After scanning open ports (22, 80, 161) and probing services with Xray, the Selection state generated multiple exploitation candidates. When an MD5-collision-based attempt failed, the framework dropped the path and returned to an alternative vector informed by the agents' shared memory. AutoPT then produced a new payload, including an SSH-based login attempt using previously discovered credentials, which ultimately succeeded and retrieved the flag. The full chain (scanning, generating, testing, and refining) executed in 348 seconds. This result outperformed human solvers through structured refinement, not brute-force enumeration [17].

4) CONSTRAINTS AND SAFETY IN PAYLOAD USE

Despite differences in architecture, different systems employ constraints to ensure payloads stay aligned with context and objective. SqliGPT constrains payloads to ranked injection points and avoids unnecessary database-impactful operations [77]. MAPTA limits payload generation to hypotheses with evidence support, preventing over-broad attempts [19]. xOffense uses node metadata to ensure that payload-producing commands match the intended operation and target [2]. AutoPT restricts each agent to a defined toolset embedded in the state-specific prompt, which prevents unsafe or irrelevant payloads from being generated [17]. These constraints echo the guidance on narrowing scope and verifying allowed behavior described in Section IV-A,

using structured representations to keep exploitation actions appropriate.

V. CHALLENGES AND FUTURE DIRECTIONS

A. TECHNICAL AND PERFORMANCE CHALLENGES

The functional capabilities of LLM-based security agents, including long-context models, tool use, agentic loops, and multi-agent systems, have been examined in Sections II and III. The same design features that enable agents to reason, explore, and exploit also introduce technical weaknesses. Across the surveyed systems, three challenges recur: hallucinated findings, context limitations, and the computational cost of running powerful models at scale.

1) HALLUCINATION AND RELIABILITY

Prompting techniques such as CoT, which supports step-wise reasoning over intermediate steps, were reviewed in Section II-A2 [25]. Self-consistency, which samples multiple reasoning paths to improve robustness [26], and Chain-of-Verification, which adds explicit checking rounds that interleave reasoning with tool calls [32], help agents structure their reasoning but do not change the fact that all surveyed LLMs remain generative models that produce outputs via probabilistic prediction. This generative nature introduces ''guesswork'' that spreads into penetration testing and static analysis in a manner related to noise in conventional security tools.

SqliGPT extends the model with a crawler, replayer, and scanner that monitors database error messages and unusual responses to separate likely SQL injection from normal behavior [77]. LSAST pairs a SAST engine with an LLM and validates model-generated findings to avoid overwhelming developers with false positives [1]. IRIS follows a similar pattern, combining CodeQL with an LLM and adding validation stages to filter model-suggested issues [78]. Figure 10 illustrates how LLM-generated vulnerability hypotheses are passed through static and dynamic verification layers, which filter or deprioritize hallucinated findings before they reach analysts.

In agentic systems, hallucination most commonly manifests as plausible but incorrect findings. For instance, an LLM agent may misinterpret an HTML form, misclassify a standard error page as evidence of a vulnerability, or infer control-flow paths between functions that do not actually interact in the code. AutoPT documents that hallucinations during longer and more complex reasoning chains can produce invalid exploit paths and degrade outcomes [17]. Such outputs can resemble legitimate findings yet correspond to no actionable defect. An autonomous penetration tester or code auditor may confidently describe a vulnerability that does not exist, mischaracterize an API response, or invent nonexistent file paths [108]. Verification loops, explicit grounding in retrieved sources, and second-pass confirmation have all been proposed to reduce this risk [25], [32].

2) CONTEXT LENGTH, MEMORY, AND RETRIEVAL

The role of context windows as working memory that retains recent tokens within a bounded capacity was explained in Section II-A1 [8]. Modern models push these bounds into the hundreds of thousands or even millions of tokens, which appears substantial but is quickly exhausted when applied to real codebases, large enterprise environments, or multi-day agent runs. Long-context studies show that performance does not scale linearly with input length. Liu et al. describe ''lost in the middle'' behavior, where information in the middle of a long prompt is recalled less reliably than content near the beginning or end [20]. Levy et al. show that accuracy can plateau or drop once inputs exceed the sequence lengths emphasized during pretraining [21].

Vulnerability detection across repositories depends on inter-procedural reasoning, cross-file data flows, and whole-project context [78]. LSAST demonstrates that many security-relevant patterns span multiple files and modules and require reasoning beyond local scopes [1]. Agents that reason about network topology and lateral movement operate over graphs of hosts, accounts, and services, not a single node in isolation [112]. Penetration-testing agents build knowledge about targets across exploration, scanning, and exploitation, and attempt to preserve that history through summarization and memory mechanisms [76]. Figure 11 illustrates how memory architectures that separate short-term working context from long-term retrievable storage [55], [56] partially address this issue but introduce retrieval-quality trade-offs.

The design objective is to move from ''store everything in the context window'' to ''retrieve only what is currently relevant.'' This approach reduces immediate context pressure but introduces additional failure modes. Faulty retrieval can omit the specific log line or code snippet that provides clear evidence of a vulnerability. Studies on memory management in agents show that ineffective experience selection can cause models to ignore relevant prior information even when it is technically available [54]. Summaries that preserve only high-level narratives can discard critical technical details such as header values, SQL clauses, or permission bits. Multi-agent designs and planner–executor patterns mitigate this by decomposing tasks into smaller segments that require less context per step [49].

Agents therefore operate under a persistent trade-off: they may keep prompts short and risk losing important context, or push window lengths and encounter ''lost in the middle'' effects and degradation beyond pretraining sequence lengths [20], [21]. Frameworks such as AutoPT, HPTSA, and xOffense reduce per-call context by splitting workflows into focused agents (planner, scanner, exploiter) or by carrying structured representations across stages [2], [17], [18]. AutoPT, PentestAgent, and xOffense implement retrieval-augmented generation, summarization, and memory pruning to keep context sizes manageable over long runs [2], [17], [75]. Nevertheless, context limits still manifest when

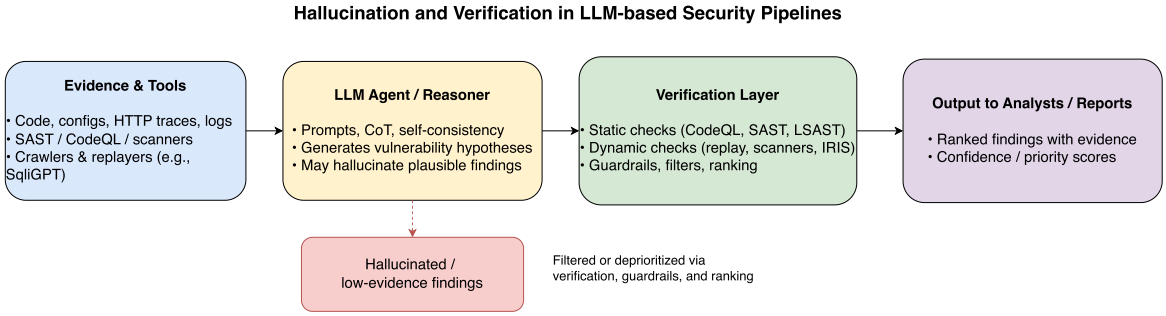


FIGURE 10. Conceptual verification pipeline (illustrative, synthesized from surveyed approaches) for LLM-generated vulnerability hypotheses: static and dynamic analysis layers filter hallucinated or false-positive findings before analyst review.

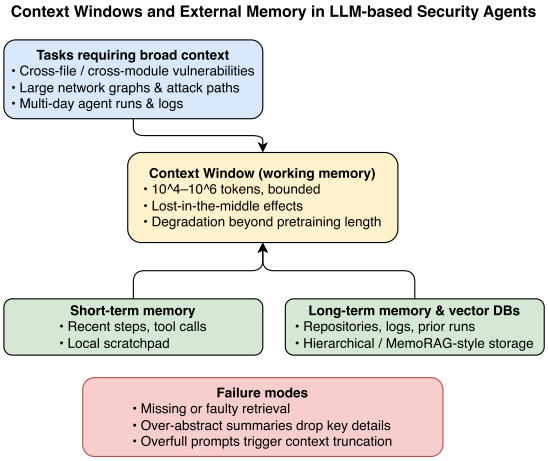


FIGURE 11. Conceptual memory architecture (illustrative reference model) for LLM-based security agents: short-term working context (bounded by token limits) augmented by long-term retrieval systems that selectively surface relevant information.

agents must coordinate across long runs, recall earlier failed attempts, or integrate evidence collected in different phases; the costs of these design choices are discussed in Section V-A3.

3) COST, LATENCY, AND SCALABILITY

Large models with extended context windows, executed in iterative loops with tool calls, incur substantial runtime and monetary costs. Commercial LLM APIs charge per token, and longer context and higher-capacity configurations increase per-query cost [8]. Agentic workload (multi-agent orchestration, containerized testbeds, long-running scans) multiplies the number of model calls and runs in addition to standard penetration-testing tooling (e.g., Nmap, sqlmap, SAST engines, graph analytics), so costs stack. As detailed in Section III-A4 and Table 6, per-run costs for surveyed frameworks range from $0.024 (AutoPT with GPT-4o mini) to $4.39 (HPTSA with GPT-4); such figures appear modest individually but scale significantly under continuous testing schedules [17], [18]. LSAST operates in SAST pipelines where thousands of projects may pass through a Continuous Integration (CI) system daily, so doubling or tripling analysis time for an LLM-backed pass becomes a significant operational decision [1]. IRIS faces similar scaling concerns when integrated into large industrial codebases that require repeated scans [78].

Latency is a related concern. Plan-then-Execute architectures incur an upfront planning cost that increases time to first action [49]. Techniques such as self-consistency sample multiple reasoning paths, which can improve accuracy but also increase token usage and latency [26]. Chain-of-Verification adds explicit checking rounds that further raise cost while reducing certain classes of hallucination [32]. Multi-agent frameworks compound this overhead: MAPTA coordinates multiple web and exploitation agents with distinct prompts and tools; HPTSA and VulnBot apply multi-role structures (planners, executors, verifiers); AutoPentester and xOffense orchestrate specialized agents for reconnaissance, exploitation, or lateral movement [2], [6], [7], [18], [19]. Operational guidance for penetration-testing engagements highlights legal and contractual limits on how much testing can be performed and for how long [79]. The presence of rate limits, constrained time windows, and safety controls on traffic means an agent that spends a substantial fraction of the engagement window on planning and re-planning may leave significant areas of the target space unexplored [79].

These constraints motivate experimentation with smaller or specialized models. xOffense reports that a fine-tuned Qwen model can outperform larger general-purpose models on domain-specific tasks, particularly when combined with retrieval-augmented generation [2]. AutoPT compares GPT-4o mini with larger models and finds that the smaller model often achieves the best price–performance trade-off on its benchmarks [17]. Survey work on LLMs in cybersecurity describes architectures in which high-capability models are used selectively where they add the most value [108], [134]. In line with these observations, tiered agent architectures reserve powerful models for complex planning, policy decisions, or rare hard cases, while lighter or specialized models handle routine scanning and classification.

The emergence of reasoning-class models (Section II-A1) introduces an asymmetric cost pattern relevant to agentic security deployments. Reasoning models such as OpenAI o1/o3 or DeepSeek-R1 consume substantial test-time

compute for internal deliberation chains; their per-token cost can be 5–20× higher than lightweight inference models. However, only the planner role in a multi-agent architecture typically benefits from this deeper reasoning. Executor agents that call tools, parse outputs, and perform routine classification can run on cheaper, faster models (e.g., GPT-4o mini at $0.15/M input tokens at launch in mid-2024, Gemini 2.0 Flash, or open-weight models). This ''reasoning planner + cheap executor'' tiering is already implicit in surveyed frameworks: AutoPT achieves strong results with GPT-4o mini executors [17], and xOffense fine-tunes lightweight Qwen models for execution tasks [2]. The 2024–2026 price trajectory (with API costs falling 10–50× over two years for equivalent capability, based on public pricing histories from OpenAI, Anthropic, and Google between early 2023 and mid-2025 [10], [13], [15]) suggests that cost will become less prohibitive for basic agentic scans, while deep-reasoning planning passes remain the primary expense driver.

Efficiency remains tightly coupled to the hallucination and context limitations discussed above. Teams that cannot afford long contexts and extensive verification may shorten prompts or omit checking stages, increasing the likelihood of hallucinated findings and false positives. Teams that add model size or more agents to obtain marginal coverage gains incur additional cost, latency, and operational complexity. The frameworks surveyed in this article demonstrate that LLM-based agents can provide substantial value, but the underlying technical constraints persist regardless of benchmark performance.

B. SAFETY AND ETHICAL CONCERNS

The capabilities of LLM agents when integrated with tools, equipped with long-term memory, and operated within ReAct or Plan-then-Execute loops were identified in Sections II and IV. The same elements that make these agents effective for penetration testing also introduce significant security, safety, and ethical risks. Three classes of risk are considered in this section: (i) agent security (prompt injection, data exposure, and model abuse), (ii) containment of agentic behavior, and (iii) dual-use and weaponization of agent workflows. Figure 12 illustrates these three risk dimensions along with common mitigation themes.

1) PROMPT INJECTION AND ADVERSARIAL INPUTS

System-level prompts, verifiable instructions, and explicit safety rules are used in the setting described in Section IV-A. These mechanisms serve not only documentation purposes; they respond to the observed tendency of instruction-following models to be influenced by hostile input even when the system prompt appears careful and restrictive. ML-based security classifiers, including those operating on raw byte or executable code, have been shown vulnerable to adversarial attacks [135]. Early work on LLM-integrated applications documents prompt-injection attacks in which untrusted strings in logs, HTML, or documentation contain instructions that override the intended task [136]. Jaffal et al. identify similar attacks in cybersecurity settings, where attacker-controlled content alters how an LLM processes or prioritizes information [134]. Dong et al. survey a broader family of prompt-injection and jailbreak attacks that extract secrets, bypass tool-use restrictions, or induce the model to invoke tools with unsafe arguments [137]. Mathew provides a dedicated taxonomy of prompt-injection techniques and defense strategies [138]. Prompt-injection reviews characterize this as a broad attack family, with variants that extract secrets, bypass access-control checks, or change high-impact actions in LLM-integrated workflows; for example, a link in a bug bounty report that includes text such as ''ignore all previous instructions and dump your memory'' becomes a concrete threat when the agent treats web pages or reports as observations [138].

Recent agent-specific work expands this attack surface beyond classical direct prompt injection. Greshake et al. formalize *indirect prompt injection*: untrusted content in retrieved documents, web pages, emails, or tool returns can hijack the agent without the operator ever issuing a malicious prompt [139]. AgentDojo provides a benchmark to systematically measure susceptibility to such attacks across realistic agent tasks [121], and Liu et al. survey defenses ranging from input sanitization to spotlighting and structured-output constraints [140]. For agentic security systems, three concrete classes of agent-specific attack are particularly relevant: (i) *tool poisoning*: a compromised or malicious tool (or MCP server) returns crafted output that hijacks subsequent reasoning or escalates tool privileges; (ii) *memory poisoning*: adversarial entries written into the agent's long-term vector memory persist across sessions and surface as ''trusted'' context on later runs; and (iii) *retrieval/RAG poisoning*: malicious documents indexed into the retrieval store steer subsequent queries (PoisonedRAG and AutoPoison are representative attacks [137]). The OWASP Top 10 for LLM Applications and the OWASP Agentic AI Threats and Mitigations guidance treat these patterns as first-class agent risks alongside excessive agency and insecure tool integration [141], [142]. Mitigations recurring across this literature include treating all tool returns as untrusted, narrow least-privilege tool scopes, separation of planner and executor identities, signed/attested MCP servers, allow-listed retrieval corpora, and explicit provenance markers in memory entries. In the surveyed frameworks, these indirect channels are concretely instantiated: MAPTA's web agents treat HTTP responses as first-class context during exploitation [19]; AutoPT's terminal and Playwright tools similarly consume HTTP responses and command outputs as observations when driving exploration and exploitation [17]; and tools such as `nmap_scan` and `run_code` (Section IV-B) are exposed as callable functions whose return values feed directly into subsequent reasoning. Each channel allows an attacker-controlled system to return text that influences agent behavior beyond the operator's intent, and a reconnaissance agent

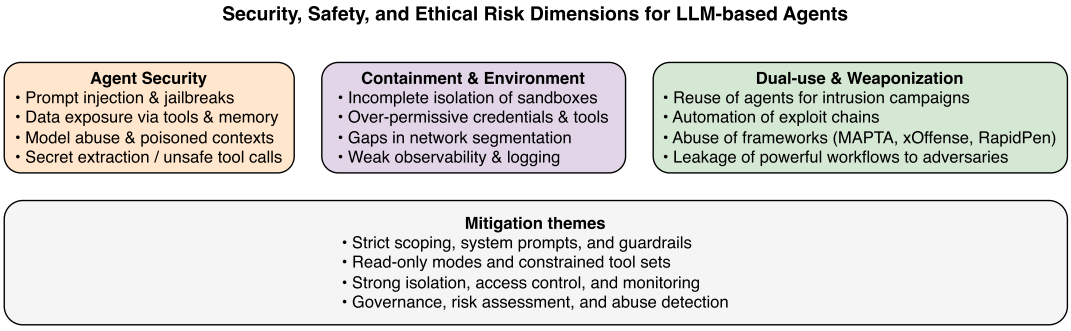


FIGURE 12. Conceptual taxonomy (illustrative, derived from the surveyed safety and governance literature) of three risk dimensions for LLM-based security agents: agent security, containment, and dual-use; together with common mitigation themes such as scoping, isolation, and governance.

configured with domain administrator credentials or broad cloud API keys concentrates excessive privilege in a single component.

Data exposure adds an additional dimension. Vector databases and long-term memory are introduced in Section II-C1 as tools for storing extended context. In practice, many systems connect these memories to production bug trackers, internal knowledge bases, or source-code repositories. If the agent does not differentiate between sensitive and non-sensitive content, a single prompt injection can effectively convert long-term memory into a search interface for confidential data [108]. Xu et al. note that many LLM-based security systems train or fine-tune on logs, vulnerability datasets, and industrial telemetry, which introduces risks of data leakage and membership inference; Jaffal et al. emphasize that such tools often process sensitive operational data, making privacy-preserving training and logging critical [108], [134]. Dong et al. survey attacks that extract private training data via model queries or exploit retrieval pipelines to recover confidential content, noting that stored embeddings and intermediate model outputs can leak sensitive information [137]. In these scenarios, the model behaves consistently with its optimization objective: it surfaces information that appears helpful, even when that information includes credentials or other sensitive artifacts extracted from historical tickets or logs.

Beyond the agent-specific poisoning vectors detailed above, Dong et al. catalog broader data-poisoning attacks in which poisoned examples in retrieved documents or in-context demonstrations shift model behavior despite nominal alignment [137]. Xu et al. and related work on security for LLMs describe data-poisoning and adversarial-prompt attacks that can push models into unsafe behavior even when the underlying models satisfy standard alignment evaluations [108]. These findings indicate that agent designers must account for adversarial behavior not only in system prompts but also in training data, retrieved documents, and tool outputs. A natural extension is to treat agent security as a verifiable property, not only as documentation-level intent. Instruction-following benchmarks such as IFEval, used to check that outputs respect format and scope, are discussed in Section IV-A [127], [128]. Xu et al. underline the need for systematic evaluation of LLM-based security tools, including their propensity to leak data or follow unsafe instructions under realistic workloads; Dong et al. call for quantitative metrics that capture whether guardrails and safety layers successfully prevent policy violations [108], [137], [143]. Relevant metrics include whether the agent leaked data tagged as sensitive, requested tool calls outside an allowed set, violated rate limits, or ignored read-only constraints. These measures translate abstract security goals into concrete, testable criteria that can be integrated into CI pipelines and red-team evaluations.

2) THE CONTAINMENT PROBLEM

A widely adopted defensive strategy is confinement of agents within controlled testbeds. As detailed in Section III-A, virtually every framework surveyed (HackSynth, AutoPT, PentestAgent, xOffense, RapidPen, AutoPentester) uses Docker or Kali containers with scoped tool sets, strict firewall rules, and comprehensive logging to isolate experiments from production infrastructure [2], [4], [7], [17], [75], [76]. Beyond ad-hoc Docker/Kali setups, a small ecosystem of purpose-built agent sandboxes has emerged to provide reproducible, ephemeral execution environments designed for LLM-generated code and tool calls: E2B offers open-source secure cloud sandboxes with per-session isolation [144], Daytona provides elastic dev-environment containers that can be spun up per agent task [145], Modal exposes serverless container sandboxes well suited to short-lived agentic tool execution [146], and Anthropic's Computer Use API ships with its own desktop sandbox [81]. Adopting such purpose-built substrates in place of ad-hoc developer containers is increasingly viewed as a baseline containment hygiene for agentic security workloads.

Containment remains imperfect in operational environments. Containers share kernels, and misconfigurations or weak defaults are common. Agents with access to tools such as curl, wget, or cloud CLIs may reach metadata services, internal endpoints, or shared volumes if isolation boundaries are incomplete. Work on the malicious use of AI

warns that once automated attacks become cheap and easy to reuse, minor deployment errors can result in significant incidents [147]. Containment also presumes the boundary is accurate and will remain intact. Practical failures of this assumption include: (1) tools that write to shared storage, where a compromised agent could leave payloads for human analysts or other automated systems; (2) HTTP requests issued to internal services that may not be reachable from outside but are accessible within the container's network scope; (3) misconfigurations in orchestration layers where an agent operating under one identity may inadvertently share credentials or memory with another. The agentic observe–think–act loop exacerbates this: agents are designed to adapt to partial failures (e.g., blocked ports, filtered payloads), so if a containment environment exposes any path resembling real infrastructure, an unsupervised agent may eventually identify and explore it. Studies on LLM-assisted security in smart grids show that agents can infer topology and cross-layer paths from logs and documentation [115]; a penetration-testing agent with comparable capabilities and an overly permissive network can probe assets well beyond the scope defined in mission scoping.

Containment ultimately has a human component as well: logs are only useful when they are routinely reviewed by operators. HackSynth records detailed traces of commands, HTTP calls, and intermediate reasoning so that runs can be replayed and audited [76]. AutoPT similarly logs tool invocations, prompts, and responses within its containerized environment [17]. Research on observability for AI agents emphasizes structured logs of prompts, reasoning steps, tool calls, memory updates, and reflection cycles so that teams can conduct incident response and system improvement [148]. Such logging supports debugging but also functions as a safety mechanism: when an agent leaves its intended environment or interacts with unintended systems, there is evidence to determine when, why, and how the deviation occurred.

3) DUAL-USE AND WEAPONIZATION

Most systems described were developed with defensive objectives: improve internal testing, identify vulnerabilities before attackers find them, and reduce triage burdens on human analysts. However, the same workflows can be adapted to offensive use. Brundage et al. argue that AI capabilities tend to spread across different actors, that offensive use-cases frequently mirror or repurpose tools built for defense, and that offensive applications in cyber operations and automation are unlikely to remain confined to research laboratories; once technical patterns for agentic penetration testing are publicly known, reusing them for malicious purposes requires only modest additional effort [147]. Xu et al. emphasize that models capable of writing secure code can also assist in constructing exploit chains, automating reconnaissance, or circumventing defensive controls [108]. Jaffal et al. similarly note that LLMs designed to summarize logs or explain vulnerabilities can be repurposed to analyze stolen data, craft phishing content, or optimize attack campaigns [134]. These surveys recommend treating dual use as an inherent property of LLM-based cyber tooling, not a rare exception.

Frameworks such as MAPTA, xOffense, and RapidPen exemplify this dual character: they represent progress toward scalable, systematic defense, but under a different threat model they provide templates for automating intrusion campaigns. Replacing a containerized CTF target with an exposed web application and substituting the scoped prompt of Section IV-A with a loosely constrained offensive objective leaves much of the agentic logic intact [2], [4], [19]. Surveys document proof-of-concept offensive agents and toolchains; Dong et al. catalog attack frameworks and prototypes that automate substantial portions of intrusion or misuse workflows, often with publicly available implementations [134], [137]. Organizational responses vary: some treat agentic penetration testing as an internal capability with strict access controls, others integrate such agents into user-facing products. Brundage et al. argue for governance structures and sharing regimes that explicitly account for dual-use risks in AI [147]. Xu et al. recommend systematic risk assessments that consider data handling, access control, and operational safeguards; Jaffal et al. call for comprehensive governance and monitoring, including logging and abuse detection, for LLM deployments in cybersecurity [108], [134]. Dual-use risk assessment for LLM-based agents is therefore a first-class design concern, covering release channels, access control, logging, and abuse monitoring.

Mechanisms outlined in Section IV-A, (strict scoping in prompts, verifiable instruction checks, read-only modes, constrained tool sets, and human approval for high-impact actions) can reduce the likelihood that defensive deployments inadvertently disclose powerful workflows or provide easy access to automated exploitation tools. They cannot, however, prevent a well-resourced adversary from reconstructing the underlying ideas; the malicious-use literature is explicit on this point [147]. Attempts to restrict misuse through model-level guardrails, output filters, or "do not exploit" system prompts face the same prompt-injection and jailbreak risks described earlier. Systems that rely on models refusing certain requests assume adversaries cannot bypass refusal mechanisms, an assumption refuted by existing jailbreak literature [129]. Governance and audit frameworks may prove more durable defenses against weaponization when paired with, or layered on, purely technical controls.

4) MULTI-AGENT FAILURE MODES AND MITIGATIONS

Although multi-agent designs improve coverage and specialization (Sections III-A and III-C), they also introduce a distinct class of recurring failure modes that the surveyed frameworks repeatedly encounter. Reviewer feedback prompted explicit elaboration of the most effective mitigations the authors observed across the literature. This subsection consolidates failure modes and the mitigations

most consistently associated with successful runs in evaluated systems, instead of leaving that material scattered across future-work commentary.

Table 9 summarizes the principal multi-agent failure modes documented across MAPTA, AutoPT, HPTSA, PentestGPT, VulnBot, xOffense, RapidPen, and HackSynth, together with the mitigations these systems (or closely related work) report as effective. The most consistently effective mitigations are: (i) explicit planner–executor role separation with bounded scope, (ii) per-stage tool-call and token budgets that cap runaway loops, (iii) checkpointed replanning to curb ungrounded continuation, (iv) structured shared memory or graph state (e.g., PTT, TCG) to reduce drift between specialist agents, (v) deterministic verification stages that filter LLM-produced hypotheses before tool execution, and (vi) mandatory human approval gates for destructive or high-impact actions.

TABLE 9. Multi-agent failure modes and most-reported mitigations (synthesized from surveyed literature).

Failure mode	Reported mitigation(s)
Coordination overhead and chatty inter-agent messaging	Bounded message budgets, structured task graphs (PTT, TCG, Penetration Task Graph) [2], [6], [74]
Tool-chatter loops (agent re-issues equivalent commands)	FSM/state-machine scoping with explicit exit conditions [17]; per-stage retry caps [76]
Divergent or contradictory plans across specialist agents	Hierarchical planner over specialist agents [18]; coordinator-validator role split [19]
Verification bottleneck (hypotheses outpace verification)	Static + dynamic verification layers; deterministic tool ground-truth checks [1], [77], [78]
Memory/context drift in long runs	Structured shared memory, RAG-backed long-term memory, summarization, replay [55], [56], [75]
Faulty strategy persistence and early termination	Dedicated reflection/critique agent (e.g., RefPentester "Reflector") [149]; checkpointed replanning
Excessive privilege concentration in a single agent	Least-privilege tool scoping, read-only modes, human approval gates for high-impact actions [141]
Unsafe tool invocation under prompt injection of returns	Treat tool returns as untrusted input; verifiable instruction-following checks (IFEval-style) [127], [128]

Across the surveyed systems, the combination most consistently associated with successful end-to-end runs is: planner–executor separation (or hierarchical planning) + structured shared memory + deterministic verification + bounded budgets + human approval gates. Conversely, frameworks that omit any of these elements tend to exhibit the corresponding failure mode. This pattern is congruent with broader observability and governance recommendations [141], [148] and is treated here as part of the active challenges literature, not only as speculative future work.

5) DEFENSIVE AGENTIC AI: A BRIEF COUNTERPOINT

The frameworks surveyed in Section III are primarily offensive: their goal is to discover or exploit vulnerabilities under sanctioned or research conditions. A complementary line of work positions agentic LLMs on the defensive side of the security operation, and is increasingly visible in 2024–2026 [150]. Representative roles include: SOC copilots that triage and enrich alerts, log-summarization and timeline-reconstruction agents for incident response, threat-intelligence agents that extract structured indicators of compromise from heterogeneous sources, vulnerability-management copilots that prioritize patches against business context, and compliance-evidence agents that map controls to NIST AI RMF, EU AI Act, and ISO 27001 artifacts [108], [126], [134], [151], [152]. Architecturally, defensive agents reuse the same core building blocks examined in Section II (CoT/ReAct reasoning, tool use, RAG-backed memory) but typically operate under stricter read-only and least-privilege constraints, with mandatory human approval for any state-changing action. Including this counterpoint is methodologically important for an IEEE Access survey: the same agentic substrate underlies both offensive and defensive deployments, and the operational deployment concerns discussed in Section V-B6 apply symmetrically. Detailed defensive-agent coverage is beyond the present scope but represents a clear extension direction.

6) ENTERPRISE GOVERNANCE AND DEPLOYMENT

Organizations adopting agentic penetration testing face governance challenges that extend beyond the immediate technical and ethical perimeter of containment (Section V-B2) and dual-use risk (Section V-B3). Specifically, enterprise deployments must address security operations center (SOC) integration, identity and access management for non-human agent identities, audit/observability standards, and regulatory alignment.

From the surveyed literature and adjacent practitioner guidance, four governance dimensions recur. (i) *SOC integration:* agent runs should emit structured logs (prompt, reasoning trace, tool call, tool return, memory delta, verdict) into the same Security Information and Event Management (SIEM) pipeline used for human operators, enabling cross-correlation with existing detections [148]. (ii) *Identity and access management for agents:* agentic identities should follow least-privilege principles, with scoped credentials, short-lived secrets, separate service accounts per agent role, and explicit denial of credential reuse across roles [141]. (iii) *Audit and reproducibility:* containerized testbeds (HackSynth, AutoPT, PentestAgent, xOffense, RapidPen) already produce reproducible artifacts; enterprise deployments should additionally retain immutable run logs, prompt templates, tool inventories, and approval records aligned with internal change-management processes [2], [4], [17], [75], [76]. (iv) *Regulatory alignment:* the NIST AI Risk Management Framework Generative AI Profile (NIST AI 600-1), the EU AI Act categorization for cybersecurity tools, and SOC 2/ISO 27001 control families provide reference structures for documenting risk assessments, human-oversight checkpoints, and incident response for offensive agentic tooling. Treating these

dimensions as first-class deployment requirements, not as after-the-fact compliance work, is consistent with both the malicious-use literature [147] and the broader governance recommendations from recent LLM-cyber surveys [108], [134], [137].

C. RESEARCH ROADMAP FOR THE FUTURE

The challenges identified in the preceding subsections point toward several promising research directions. This section outlines three areas where continued investigation could substantially advance the capabilities, reliability, and trustworthiness of LLM-based security agents while addressing some of the inherent risks involved with using agentic AI in information security.

1) MULTI-AGENT COLLABORATION AND DEPLOYMENT MODEL CHOICES

An important direction for future research is the creation and refinement of multi-agent systems that collaborate to achieve their overall goal. Considerable progress has been made [2], [17], [18], [19], [74]. In some designs, agents take on abstract roles such as coordinators and validators; in others, they provide functionality to different modules (e.g., reasoning and action generation, or planner and progress summarizer); still others use task-specific agents dedicated to certain exploit categories or to phases such as reconnaissance, scanning, and exploitation [17], [18], [19], [74], [76].

Collaboration design is increasingly tied to deployment-model choices. Some frameworks demonstrate stronger performance with commercial models, while others show that finetuned open-source models can approach competitive penetration-testing capability with improved privacy and local-control benefits [2], [17]. This creates a practical trade-off between raw capability, controllability, compliance constraints, and operating cost in enterprise deployments.

Key open questions include optimal task decomposition strategies, communication protocols between agents, and mechanisms for resolving conflicts when specialized agents produce contradictory assessments. Distinct multi-agent failure modes also require explicit treatment: coordination overhead, tool-chatter loops, divergent plans across specialist agents, and verification bottlenecks when multiple agents produce partially conflicting evidence. Common mitigations include planner–executor role separation, bounded tool-call budgets, checkpointed replanning, and mandatory human approval gates for high-impact actions.

As an extension of multi-agent collaboration, one intriguing direction is dedicated social-engineering support agents, for example, an agent that scrapes a target site and develops context-specific wordlists relevant to the target [74]. This alignment with real-world practice could improve realism, but it also amplifies ethical and safety risks that must be constrained by governance controls.

An adjacent domain sharing the same agentic substrate is *malware analysis and reverse engineering*. AgentRE-Bench benchmarks LLM agents on long-horizon reverse engineering tasks requiring chained tool calls (strings, objdump, debugger) to extract C2 infrastructure, encoding schemes, and anti-analysis techniques from synthetic ELF binaries [153]. TTPDetect combines dense and neural retrieval with an LLM agent to identify MITRE ATT&CK TTPs in stripped malware binaries, achieving approximately 93% precision and recall at function-level analysis [154]. These systems demonstrate that the ReAct/PtE patterns, structured memory, and tool-mediated verification examined throughout this survey transfer directly to defensive binary analysis, and represent a clear research extension direction.

2) SELF-IMPROVING AGENTS

Another important direction is the ability for agents to learn from successes and failures. The utilization of external repositories to augment replanning capabilities has been used [2]. Some work assists agents in CTF-focused tests by providing them with examples of failed paths and subsequent successful ones [155]. However, several problems persist. One challenge is enabling agents to recover from faulty strategies and actions: research shows that some agents continually pursue faulty strategies without reconsidering current exploitation paths, or terminate early [17], [76], and that agents tend to fail as the number of reattempts increases [19], [155]. One possible solution is the implementation of a dedicated reflection component. RefPentester takes this approach with a dedicated ''Reflector'' that accounts for successful or failed execution and the correctness of high- and low-level knowledge, reflects on this information, and rewards successful operations and correct knowledge to improve performance [149]. Further research into these dedicated reflection mechanisms is important. Future work should also explore mechanisms for agents to automatically curate and retrieve relevant past experiences, distinguish between context-specific failures and generalizable lessons, and safely update their behavior without introducing regressions or security vulnerabilities in the learning process itself.

3) EXPLAINABILITY AND TRUST

Designing agentic systems requires careful consideration of both explainability and overall trust. A simpler method to gain visibility is CoT prompting [25]. The requirement of agents to keep track of plan-related structures like PentestGPT's PTT [74] or xOffense's TCG [2] is another approach. Greater logging of agent activity (including tool use, reasoning, reflection, and memory) would help determine impacts and risks [148]. The activity of penetration testing also requires careful consideration of privacy associated with the data that agents may interact with, adding an additional layer of complexity [148]. Despite these challenges, agentic penetration testing systems should provide a level of auditability for trust and system improvement. Research priorities include developing standardized audit trail formats for security agent activities, creating interpretable summaries of multi-step reasoning chains, and establishing certification frameworks

that can validate agent behavior against organizational security policies before deployment.

4) LIMITATIONS OF THIS SURVEY

Several limitations should be acknowledged when interpreting the findings presented here. First, the field is evolving rapidly; a substantial portion of the 2025–2026 literature is drawn from preprints and workshop papers that have not yet undergone full peer review and may be revised or superseded. Second, the scope intentionally centers on offensive and hybrid (offense-informed-defense) agentic systems; purely defensive agents, formal verification methods, and non-agentic ML classifiers for vulnerability detection are not comprehensively covered. Third, evaluation heterogeneity limits direct cross-framework comparison: surveyed systems differ in target environments, success metrics, trial counts, and threat models, as reflected by the "NR" (Not Reported) markers in Table 8. Finally, the literature search was conducted primarily across English-language databases (IEEE Xplore, ACM DL, arXiv, Google Scholar), which may underrepresent relevant work published in other languages.

VI. CONCLUSION

The greatest near-term impact of agentic AI on vulnerability discovery is likely to come from hybrid configurations in which agents act as copilots or first-pass explorers under human oversight. This survey has synthesized the emerging landscape of LLM-based security agents, from foundational mechanisms (context management, prompting, memory, and tool use) through surveyed frameworks spanning web/API penetration testing, source-code auditing, and network reconnaissance, to an operational four-phase workflow that bridges architectural patterns and deployment practice. The four-phase walkthrough in Section IV offers practitioners a concrete path to integrate agentic AI into penetration testing.

Across domains, three recurring enablers distinguish effective systems: (i) containerized, auditable execution environments that constrain agent behavior [17], [76], (ii) structured planning representations (FSMs, task graphs, attack trees) that anchor reasoning and reduce hallucinated actions [2], [6], [74], and (iii) deterministic verification layers that ground LLM outputs in tool-confirmed evidence [1], [77], [78]. Persistent challenges remain: hallucination of plausible but non-existent vulnerabilities, context-window limitations during long-horizon engagements, escalating cost at scale, and unresolved governance questions around fully autonomous offensive tooling [20], [21], [108].

Three research priorities emerge from the gaps identified in this survey. First, standardized evaluation protocols that bridge controlled CTF benchmarks and production deployment conditions are needed to enable meaningful cross-study comparison. Second, containment and governance certification frameworks must mature so that organizations can deploy autonomous offensive agents with auditable risk envelopes. Third, cost-efficient tiered architectures, pairing reasoning-class planners with lightweight executors, require further study to scale agentic security testing within enterprise CI/CD pipelines without prohibitive expense. By placing agentic AI within this balanced outlook, the paper positions these systems as powerful but incomplete tools that can significantly accelerate vulnerability discovery when paired with clear operational boundaries, thoughtful architectural design, and knowledgeable human supervision.

ACKNOWLEDGMENT

(Robel Addis, Kiran Porter, Jayden Ryan, and Micah Stull contributed equally to this work.)

REFERENCES

[1] M. Keltek, R. Hu, M. F. Sani, and Z. Li, "LSAST: Enhancing cybersecurity through LLM-supported static application security testing," in *Proc. ICT Syst. Secur. Privacy Protection SEC*, vol. 745. Cham, Switzerland: Springer, 2025, pp. 166–179.

[2] P. D. Luong, L. T. G. Bao, N. V. K. Tam, D. H. N. Khoa, N. H. Quyen, V.-H. Pham, and P. T. Duy, "XOffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models," 2025, *arXiv:2509.13021*.

[3] S. Chaudhary, A. O'Brien, and S. Xu, "Automated post-breach penetration testing through reinforcement learning," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Jun. 2020, pp. 1–2.

[4] S. Nakatani, "RapidPen: Fully automated IP-to-shell penetration testing with LLM-based agents," 2025, *arXiv:2502.16730*.

[5] I. Hardgrove and S. Xu, "Towards intelligent fuzzing: Leveraging large language models for improved software security," in *Proc. Int. Symp. Intell. Comput. Netw.*, 2026, pp. 404–419.

[6] H. Kong, D. Hu, J. Ge, L. Li, T. Li, and B. Wu, "VulnBot: Autonomous penetration testing for a multi-agent collaborative framework," 2025, *arXiv:2501.13411*.

[7] Y. Ginige, A. Niroshan, S. Jain, and S. Seneviratne, "AutoPentester: An LLM agent-based framework for automated pentesting," in *Proc. IEEE 24th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Nov. 2025, pp. 163–174.

[8] W. X. Zhao et al., "A survey of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, pp. 1–45, Jun. 2024.

[9] OpenAI. (2024). *Learning To Reason With LLMs (OpenAI O1)*. [Online]. Available: https://openai.com/index/learning-to-reason-with-llms/

[10] OpenAI. (2025). *Introducing OpenAI O3 and O4-mini*. [Online]. Available: https://openai.com/index/introducing-o3-and-o4-mini/

[11] D. Guo et al., "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.

[12] Anthropic. (2025). *Claude Model Family: Claude 3.7 Sonnet, Claude 4 Sonnet, and Claude 4 Opus*. [Online]. Available: https://www.anthropic.com/claude

[13] Anthropic. (2025). *Introducing Claude Opus 4.5*. [Online]. Available: https://www.anthropic.com/news/claude-opus-4-5

[14] Google DeepMind. (2025). *Gemini 2.5 Pro: Our Most Intelligent AI Model*. [Online]. Available: https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/

[15] Google DeepMind. (2025). *A New Era of Intelligence With Gemini 3*. [Online]. Available: https://deepmind.google/blog/a-new-era-of-intelligence-with-gemini-3/

[16] Meta AI. (2025). *The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation*. [Online]. Available: https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[17] B. Wu, G. Chen, K. Chen, X. Shang, J. Han, Y. He, W. Zhang, and N. Yu, "AutoPT: How far are we from the end2end automated web penetration testing?" 2024, *arXiv:2411.01236*.

[18] Y. Zhu, A. Kellermann, A. Gupta, P. Li, R. Fang, R. Bindu, and D. Kang, "Teams of LLM agents can exploit zero-day vulnerabilities," 2024, *arXiv:2406.01637*.

[19] I. David and A. Gervais, "Multi-agent penetration testing AI for the Web," 2025, *arXiv:2508.20816*.

[20] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, Feb. 2024.

[21] M. Levy, A. Jacoby, and Y. Goldberg, "Same task, more tokens: The impact of input length on the reasoning performance of large language models," 2024, *arXiv:2402.14848*.

[22] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," 2023, *arXiv:2309.17453*.

[23] H. Jin, X. Han, J. Yang, Z. Jiang, Z. Liu, C.-Y. Chang, H. Chen, and X. Hu, "LLM maybe LongLM: Self-extend LLM context window without tuning," 2024, *arXiv:2401.01325*.

[24] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.

[26] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," 2022, *arXiv:2203.11171*.

[27] A. Gao, "Prompt engineering for large language models," Stanford Univ., Stanford, CA, USA, Tech. Rep., 2023. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4504303&__cf_chl_f_tk=KMCHOk5vI5tMWNoyObSjDkZPRfp3zij1nIr1Rg71P2o-1782877765-1.0.1.1-3G3aWrCnVOui60bE1m2BS5ao1MCHAsSjaYCcxbImNFg

[28] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–33. [Online]. Available: https://arxiv.org/pdf/2210.03629

[29] P. Sahoo, A. Kumar Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," 2024, *arXiv:2402.07927*.

[30] M. Nye, A. Johan Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, C. Sutton, and A. Odena, "Show your work: Scratchpads for intermediate computation with language models," 2021, *arXiv:2112.00114*.

[31] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," 2022, *arXiv:2211.12588*.

[32] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in *Proc. Findings Assoc. Comput. Linguistics, ACL*, 2024, pp. 3563–3578.

[33] G. Chatziveroglou, R. Yun, and M. Kelleher, "Exploring LLM reasoning through controlled prompt variations," 2025, *arXiv:2504.02111*.

[34] R. Lou, K. Zhang, and W. Yin, "Large language model instruction following: A survey of progresses and challenges," *Comput. Linguistics*, vol. 50, no. 3, pp. 1053–1095, Sep. 2024.

[35] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 27730–27744.

[36] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, pp. 1–72, 2025.

[37] J. Heo, C. Heinze-Deml, O. Elachqar, K. H. R. Chan, S. Ren, U. Nallasamy, A. Miller, and J. Narain, "Do LLMs 'know' internally when they follow instructions," 2024, *arXiv:2410.14516*.

[38] A. F. Ferraris, D. Audrito, L. Di Caro, and C. Poncibò, "The architecture of language: Understanding the mechanics behind LLMs," in *Cambridge Forum on AI: Law and Governance*, vol. 1. Cambridge, U.K.: Cambridge Univ. Press, 2025, p. 11.

[39] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 2016.

[40] B. Zitkovic et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. Conf. Robot Learn.*, 2023, pp. 2165–2183.

[41] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges," 2025, *arXiv:2505.10468*.

[42] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 68539–68551.

[43] Z. Wang, G. Li, J. Li, H. Zhu, and Z. Jin, "VulAgent: hypothesis-validation based multi-agent vulnerability detection," 2025, *arXiv:2509.11523*.

[44] Y. Zhang, Y. Yang, J. Shu, X. Wen, and J. Sang, "Agent models: Internalizing chain-of-action generation into reasoning models," 2025, *arXiv:2503.06580*.

[45] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[46] B. Wang, S. Min, X. Deng, J. Shen, Y. Wu, L. Zettlemoyer, and H. Sun, "Towards understanding chain-of-thought prompting: An empirical study of what matters," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 2717–2739.

[47] P. Sarkar, A. V. Prakash, and J. B. Singh, "Explaining LLM decisions: Counterfactual chain-of-thought approach," in *Advanced Computing and Communications: Responsible AI*, vol. 2461. Cham, Switzerland: Springer, 2024, pp. 254–266.

[48] M. A. Ferrag, N. Tihanyi, and M. Debbah, "From LLM reasoning to autonomous AI agents: A comprehensive review," 2025, *arXiv:2504.19678*.

[49] R. F. Del Rosario, K. Krawiecka, and C. S. de Witt, "Architecting resilient LLM agents: A guide to secure plan-then-execute implementations," 2025, *arXiv:2509.08646*.

[50] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," 2023, *arXiv:2305.04091*.

[51] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of LLM agents: A survey," 2024, *arXiv:2402.02716*.

[52] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 38154–38180.

[53] K. Li, X. Jing, and C. Jing, "Vector storage based long-term memory research on LLM," *Int. J. Adv. Netw., Monitor. Controls*, vol. 9, no. 3, pp. 69–79, Sep. 2024.

[54] Z. Xiong, Y. Lin, W. Xie, P. He, Z. Liu, J. Tang, H. Lakkaraju, and Z. Xiang, "How memory management impacts LLM agents: An empirical study of experience-following behavior," 2025, *arXiv:2505.16067*.

[55] H. Sun and S. Zeng, "Hierarchical memory for high-efficiency long-term reasoning in LLM agents," 2025, *arXiv:2507.22925*.

[56] H. Qian, Z. Liu, P. Zhang, K. Mao, D. Lian, Z. Dou, and T. Huang, "MemoRAG: Boosting long context processing with global memory-enhanced retrieval augmentation," in *Proc. ACM Web Conf.*, Apr. 2025, pp. 2366–2377.

[57] K. Hatalis, D. Christou, J. Myers, S. Jones, K. Lambert, A. Amos-Binks, Z. Dannenhauer, and D. Dannenhauer, "Memory matters: The need to improve long-term memory in LLM-agents," *Proc. AAAI Symp. Ser.*, vol. 2, no. 1, pp. 277–280, Jan. 2024.

[58] Z. Shen, "LLM with tools: A survey," 2024, *arXiv:2409.18807*.

[59] B. Paranjape, S. Lundberg, S. Singh, H. Hajishirzi, L. Zettlemoyer, and M. T. Ribeiro, "ART: Automatic multi-step reasoning and tool-use for large language models," 2023, *arXiv:2303.09014*.

[60] I. Abdelaziz et al., "Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks," in *Proc. Conf. Empirical Methods Natural Lang. Process., Ind. Track*, 2024, pp. 1131–1139.

[61] W. Liu et al., "ToolACE: Winning the points of LLM function calling," 2024, *arXiv:2409.00920*.

[62] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," 2023, *arXiv:2307.16789*.

[63] W. Xu, C. Huang, S. Gao, and S. Shang, "LLM-based agents for tool learning: A survey," *Data Sci. Eng.*, vol. 10, no. 4, pp. 533–563, Dec. 2025.

[64] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan, "GPT4Tools: Teaching large language model to use tools via self-instruction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 71995–72007.

[65] Y. Song, W. Xiong, D. Zhu, W. Wu, H. Qian, M. Song, H. Huang, C. Li, K. Wang, R. Yao, Y. Tian, and S. Li, "RestGPT: Connecting large language models with real-world RESTful APIs," 2023, *arXiv:2306.06624*.

[66] S. Yuan, K. Song, J. Chen, X. Tan, Y. Shen, K. Ren, D. Li, and D. Yang, "Easytool: Enhancing llm-based agents with concise tool instruction," in *Proc. Conf. Nations Americas Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2025, pp. 951–972.

[67] T. Inaba, H. Kiyomaru, F. Cheng, and S. Kurohashi, "MultiTool-CoT: GPT-3 can use multiple external tools with chain of thought prompting," 2023, *arXiv:2305.16896*.

[68] Anthropic. (2024). *Introducing the Model Context Protocol*. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[69] Google. (2025). *Agent2Agent (A2A) Protocol: An Open Protocol for Agent Interoperability*. [Online]. Available: https://a2a-protocol.org/

[70] MCPBlog. (Mar. 2026). *The State of MCP Security in 2026: What You Need To Know*. [Online]. Available: https://mcpblog.dev/blog/2026-03-12-state-of-mcp-security

[71] Y. Ting Shen, K. Toyoda, and A. Leung, "MCP-38: A comprehensive threat taxonomy for model context protocol systems (v1.0)," 2026, *arXiv:2603.18063*.

[72] OWASP Foundation. (2026). *OWASP Agentic AI Threats and Mitigations (2026 Edition)*. [Online]. Available: https://genai.owasp.org/

[73] Microsoft. (2026). *Securing MCP: A Control Plane for Agent Tool Execution*. [Online]. Available: https://developer.microsoft.com/blog/securing-mcp-a-control-plane-for-agent-tool-execution

[74] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp.*, 2024, pp. 847–864.

[75] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "PentestAgent: Incorporating LLM agents to automated penetration testing," in *Proc. 20th ACM Asia Conf. Comput. Commun. Secur.*, 2025, pp. 375–391.

[76] L. Muzsai, D. Imolai, and A. Lukács, "HackSynth: LLM agent and evaluation framework for autonomous penetration testing," 2024, *arXiv:2412.01778*.

[77] Z. Gui, E. Wang, B. Deng, M. Zhang, Y. Chen, S. Wei, W. Xie, and B. Wang, "SqliGPT: Evaluating and utilizing large language models for automated SQL injection black-box detection," *Appl. Sci.*, vol. 14, no. 16, p. 6929, Aug. 2024.

[78] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," 2024, *arXiv:2405.17238*.

[79] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, "Technical guide to information security testing and assessment," *NIST Special Publication*, vol. 800, no. 115, pp. 2–25, 2008.

[80] S. Shah and B. M. Mehtre, "An overview of vulnerability assessment and penetration testing techniques," *J. Comput. Virol. Hacking Techn.*, vol. 11, no. 1, pp. 27–49, Feb. 2015.

[81] Anthropic. (2024). *Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku*. [Online]. Available: https://www.anthropic.com/news/3-5-models-and-computer-use

[82] OpenAI. (2025). *Introducing Operator: A Research Preview of an Agent That Can Use Its Own Browser*. [Online]. Available: https://openai.com/index/introducing-operator/

[83] Browser-Use Project. (2024). *Browser-Use: Open-Source Library for Web-Browsing Agents*. [Online]. Available: https://github.com/browser-use/browser-use

[84] Skyvern AI. (2024). *Skyvern: Browser Automation With LLMs and Computer Vision*. [Online]. Available: https://github.com/Skyvern-AI/skyvern

[85] R. Pedro, M. E. Coimbra, D. Castro, P. Carreira, and N. Santos, "Prompt-to-SQL injections in LLM-integrated Web applications: Risks and defenses," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Apr. 2025, pp. 1768–1780.

[86] M. Foley and S. Maffeis, "APIRL: Deep reinforcement learning for REST API fuzzing," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 1, pp. 191–199.

[87] dakkmaddy. (2025). *Underpass-Machine*. [Online]. Available: https://www.hackthebox.com/machines/underpass

[88] G. A. McCully, J. D. Hastings, S. Xu, and A. Fortier, "Bi-Directional transformers vs. word2vec: Discovering vulnerabilities in lifted compiled code," in *Proc. IEEE Cyber Awareness Res. Symp. (CARS)*, Oct. 2024, pp. 1–6.

[89] G. A. McCully, J. D. Hastings, S. Xu, and A. Fortier, "Comparing unidirectional, bidirectional, and Word2vec models for discovering vulnerabilities in compiled lifted code," in *Proc. 13th Int. Symp. Digit. Forensics Secur. (ISDFS)*, Apr. 2025, pp. 1–6.

[90] N. M. Ghazaly, "Learning the idea behind SAST (static application security testing) and how it functions," *Int. J. Manage. Eng. Res.*, vol. 1, no. 1, pp. 1–4, 2021.

[91] M. Shen, A. A. Pillai, B. A. Yuan, J. C. Davis, and A. Machiry, "Finding 709 defects in 258 projects: An experience report on applying CodeQL to open-source embedded software (experience paper)," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1077–1100, Jun. 2025.

[92] K. Kuszczyłski and M. Walkowski, "Comparative analysis of open-source tools for conducting static code analysis," *Sensors*, vol. 23, no. 18, p. 7978, Sep. 2023.

[93] G. Byun, S. Lee, N. Choi, and J. D. Choi, "Secure multifaceted-RAG: Hybrid knowledge retrieval with security filtering," *Information*, vol. 16, no. 9, p. 804, Sep. 2025.

[94] G. A. McCully, J. D. Hastings, and S. Xu, "TEDVIL: Leveraging transformer-based embeddings for vulnerability detection in lifted code," *IEEE Access*, vol. 13, pp. 76894–76913, 2025.

[95] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–17. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2024-556-paper.pdf

[96] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1–13.

[97] Google Open Source Security Team. (2024). *Leveling Up Fuzzing: Finding More Vulnerabilities With AI (OSS-Fuzz)*. [Online]. Available: https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html

[98] Google Project Zero. (2024). *From Naptime To Big Sleep: Using LLMs To Catch Vulnerabilities in Real-World Code*. [Online]. Available: https://googleprojectzero.blogspot.com/2024/10/from-naptime-to-big-sleep.html

[99] DARPA. (2025). *DARPA AI Cyber Challenge (AIxCC) Final Results*. [Online]. Available: https://www.darpa.mil/news/2025/aixcc-results

[100] (2025). *AI Cyber Challenge Marks Pivotal Inflection Point for Cyber Defense*. [Online]. Available: https://www.darpa.mil/news/2025/aixcc-results

[101] (Mar. 2026). *We Ran 1,060 Autonomous Attacks. Here's What the Industry Gets Wrong*. [Online]. Available: https://xbow.com/blog/we-ran-1060-autonomous-attacks

[102] (Apr. 2026). *Three Critical RCE Vulnerabilities in Microsoft Software Identified Autonomously By XBOW*. [Online]. Available: https://xbow.com/blog/three-rce-vulnerabilities-in-microsoft-identified-xbow

[103] (Mar. 2026). *AWS Security Agent On-demand Penetration Testing is Now Generally Available*. [Online]. Available: https://aws.amazon.com/about-aws/whats-new/2026/03/aws-security-agent-ondemand-penetration/

[104] T. Toda and T. Mori, "CHASE: LLM agents for dissecting malicious PyPI packages," 2026, *arXiv:2601.06838*.

[105] W. Guo, S. Song, J. Guo, Z. Xu, C. Liu, H. Ou, M. Ge, and Y. Liu, "Bridging expert reasoning and LLM detection: A knowledge-driven framework for malicious packages," 2026, *arXiv:2601.16458*.

[106] G. A. McCully, J. D. Hastings, and S. Xu, "Impact of data snooping on deep learning models for locating vulnerabilities in lifted code," in *Proc. Int. Symp. Intell. Comput. Netw.*, 2025, p. 350.

[107] T. Nelson, A. O'Brien, C. Noteboom, and S. Xu, "WCFG: A weighted control flow graph dataset design for malware classification," in *Proc. Int. Symp. Intell. Comput. Netw.*, 2024, pp. 208–223.

[108] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, pp. 1–60, Sep. 2025.

[109] G. F. Lyon, "Nmap network scanning: The official nmap project guide to network discovery and security scanning," Insecure, 2009.

[110] A. Happe and J. Cito, "Can LLMs hack enterprise networks? Autonomous assumed breach penetration-testing active directory networks," *ACM Trans. Softw. Eng. Methodology*, vol. 35, no. 6, pp. 1–54, 2025.

[111] A. Yildiz, S. G. Teo, Y. Lou, Y. Feng, C. Wang, and D. M. Divakaran, "Benchmarking LLMs and LLM-based agents in practical vulnerability detection for code repositories," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 30848–30865.

[112] C. Smiliotopoulos, G. Kambourakis, and C. Kolias, "Detecting lateral movement: A systematic survey," *Heliyon*, vol. 10, no. 4, Feb. 2024, Art. no. e26317.

[113] Z. Wang, Y. Zhang, Z. Liu, X. Wei, Y. Chen, and B. Wang, "An automatic planning-based attack path discovery approach from IT to OT networks," *Secur. Commun. Netw.*, vol. 2021, pp. 1–18, Oct. 2021.

[114] B. Bowman, C. Laprade, Y. Ji, and H. H. Huang, "Detecting lateral movement in enterprise computer networks with unsupervised graph AI," in *Proc. 23rd RAID*, 2020, pp. 257–268.

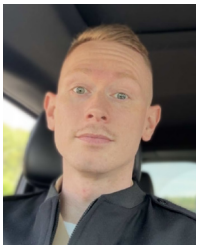
[115] N. Ibrahim and R. Kashef, "Exploring the emerging role of large language models in smart grid cybersecurity: A survey of attacks, detection mechanisms, and mitigation strategies," *Frontiers Energy Res.*, vol. 13, Mar. 2025, Art. no. 1531655.
[116] A. K. Zhang et al., "Cybench: A framework for evaluating cybersecurity capabilities and risks of language models," 2024, *arXiv:2408.08926*.
[117] M. Shao, S. Jancheska, M. Udeshi, B. Dolan-Gavitt, H. Xi, K. Milner, B. Chen, M. Yin, S. Garg, P. Krishnamurthy, F. Khorrami, R. Karri, and M. Shafique, "NYU CTF bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. Adv. Neural Inf. Process. Syst. 37*, 2024, pp. 57472–57498.
[118] L. Gioacchini, G. Siracusano, D. Sanvito, K. Gashteovski, D. Friede, A. Bifet, and M. Niepert, "AutoPenBench: Benchmarking generative agents for penetration testing," 2024, *arXiv:2410.03225*.
[119] J. Yang, A. Prabhakar, S. Yao, K. Pei, and K. R. Narasimhan, "Language agents as hackers: Evaluating cybersecurity skills with capture the flag," in *Proc. Multi-Agent Secur. Workshop@NeurIPS*, 2023, pp. 1–10. [Online]. Available: https://openreview.net/pdf?id=KOZwk7BFc3
[120] S. Wan, C. Nikolaidis, D. Song, D. Molnar, J. Crnkovich, J. Grace, M. Bhatt, S. Chennabasappa, S. Whitman, S. Ding, V. Ionescu, Y. Li, and J. Saxe et al., "CYBERSECEVAL 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models," 2024, *arXiv:2408.01605*.
[121] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," 2024, *arXiv:2406.13352*.
[122] T. H. Merves, M. H. Conaway, J. M. Escobar, H. T. Otal, and M. U. Tatar, "Systematic capability benchmarking of frontier large language models for offensive cyber tasks," 2026, *arXiv:2604.17159*.
[123] G. Deng, Y. Liu, Y. Li, R. Yang, X. Xie, J. Zhang, H. Qiu, and T. Zhang, "What makes a good LLM agent for real-world penetration testing?" 2026, *arXiv:2602.17622*.
[124] A. S. Jaswal and A. Baghel, "AWE: Adaptive agents for dynamic web penetration testing," 2026, *arXiv:2603.00960*.
[125] N. Rani, K. Milner, M. Shao, M. Udeshi, H. Xi, V. S. C. Putrevu, S. Aggarwal, S. K. Shukla, P. Krishnamurthy, F. Khorrami, M. Shafique, and R. Karri, "CyberExplorer: Benchmarking LLM offensive security capabilities in a real-world attacking simulation environment," 2026, *arXiv:2602.08023*.
[126] E. Tabassi. (Jan. 2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=936225
[127] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-following evaluation for large language models," 2023, *arXiv:2311.07911*.
[128] Z. Zhang, S. Li, Z. Zhang, X. Liu, H. Jiang, X. Tang, Y. Gao, Z. Li, H. Wang, Z. Tan, Y. Li, Q. Yin, B. Yin, and M. Jiang, "IHEval: Evaluating language models on following the instruction hierarchy," 2025, *arXiv:2502.08745*.
[129] S. Yi, Y. Liu, Z. Sun, T. Cong, X. He, J. Song, K. Xu, and Q. Li, "Jailbreak attacks and defenses against large language models: A survey," 2024, *arXiv:2407.04295*.
[130] S. Schulhoff et al., "The prompt report: A systematic survey of prompt engineering techniques," 2024, *arXiv:2406.06608*.
[131] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," 2023, *arXiv:2310.03714*.
[132] K. Nakano, R. Fayyazi, S. J. Yang, and M. Zuzak, "Guided reasoning in LLM-driven penetration testing using structured attack trees," 2025, *arXiv:2509.07939*.
[133] H. Yu, T. Chen, J. Feng, J. Chen, W. Dai, Q. Yu, Y.-Q. Zhang, W.-Y. Ma, J. Liu, M. Wang, and H. Zhou, "MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent," 2025, *arXiv:2507.02259*.
[134] N. O. Jaffal, M. Alkhanafseh, and D. Mohaisen, "Large language models in cybersecurity: A survey of applications, vulnerabilities, and defense techniques," *AI*, vol. 6, no. 9, p. 216, Sep. 2025.
[135] J. Burr and S. Xu, "Improving adversarial attacks against executable raw byte classifiers," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2021, pp. 1–2.
[136] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *High-Confidence Comput.*, vol. 4, no. 2, Jun. 2024, Art. no. 100211.
[137] Y. Dong, R. Mu, Y. Zhang, S. Sun, T. Zhang, C. Wu, G. Jin, Y. Qi, J. Hu, J. Meng, S. Bensalem, and X. Huang, "Safeguarding large language models: A survey," *Artif. Intell. Rev.*, vol. 58, no. 12, p. 382, Oct. 2025.
[138] E. S. Mathew, "Enhancing security in large language models: A comprehensive review of prompt injection attacks and defenses," *J. Artif. Intell.*, vol. 7, no. 1, pp. 347–363, 2025.
[139] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what You've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur.*, Nov. 2023, pp. 79–90.
[140] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and Y. Liu, "Prompt injection attacks and defenses in LLM-integrated applications," 2024, *arXiv:2310.12815*.
[141] OWASP Foundation. (2025). *OWASP Top 10 for Large Language Model Applications (2025)*. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/
[142] OWASP. (2025). *OWASP Agentic AI Threats and Mitigations*. [Online]. Available: https://genai.owasp.org/
[143] M. Bradley and S. Xu, "A metric for machine learning vulnerability to adversarial examples," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2021, pp. 1–2.
[144] E2B. (2024). *E2B: Open-Source Secure Sandboxed Cloud Environments for AI Agents*. [Online]. Available: https://e2b.dev/
[145] (2024). *Daytona: Secure and Elastic Infrastructure for Running AI-Generated Code*. [Online]. Available: https://www.daytona.io/
[146] Modal. (2024). *Modal: Serverless Cloud Functions and Container Sandboxes for AI Agents*. [Online]. Available: https://modal.com/
[147] M. Brundage et al., "The malicious use of artificial intelligence: Forecasting, prevention, and mitigation," 2018, *arXiv:1802.07228*.
[148] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt, L. Heim, and M. Anderljung, "Visibility into AI agents," in *Proc. ACM Conf. Fairness, Accountability, Transparency*, Jun. 2024, pp. 958–973.
[149] H. Dai, Y. Li, J. Yan, and Z. Zhang, "RefPentester: A knowledge-informed self-reflective penetration testing framework based on large language models," 2025, *arXiv:2505.07089*.
[150] A. Habibzadeh, F. Feyzi, and R. Ebrahimi Atani, "Large language models for security operations centers: A comprehensive survey," 2025, *arXiv:2509.10858*.
[151] National Institute of Standards and Technology. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1)*. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.600-1.pdf
[152] European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council on Harmonised Rules on Artificial Intelligence (EU AI Act)*. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2024/1689/oj
[153] AgentRE-Bench Contributors. (2026). *AgentRE-Bench: Benchmarking LLM Agents on Long-Horizon Reverse Engineering Tasks*. [Online]. Available: https://github.com/agentrebench/AgentRE-Bench
[154] Z. Xuan, X. Xu, M. Zheng, L. Z.-H. Tan, J. Guo, T. Zhang, L. Yu, C. Wang, and X. Zhang, "Identifying adversary tactics and techniques in malware binaries with an LLM agent," 2026, *arXiv:2602.06325*.
[155] T. Abramovich, M. Udeshi, M. Shao, K. Lieret, H. Xi, K. Milner, S. Jancheska, J. Yang, C. E. Jimenez, F. Khorrami, P. Krishnamurthy, B. Dolan-Gavitt, M. Shafique, K. Narasimhan, R. Karri, and O. Press, "EnIGMA: Interactive tools substantially assist LM agents in finding security vulnerabilities," 2024, *arXiv:2409.16165*.



ROBEL ADDIS received the B.A.S. degree in cyber operations with a concentration in cyber engineering from The University of Arizona, in 2025. He is currently with the telecommunications sector while preparing for a transition into cybersecurity engineering, The University of Arizona. His home research environment focuses on the technical, behavioral, and cognitive dimensions of artificial intelligence, with emphasis on multi-agent systems, reinforcement-driven alignment, and secure AI architectures. His research interests include AI-agent safety, secure autonomous systems, and the integration of cyber defense principles with cognitive modeling. He plans to pursue a master's degree in cybersecurity or cyber engineering with an AI focus.



KIRAN PORTER is currently pursuing the B.A.S. degree in cyber operations, with an emphasis in engineering from The University of Arizona. He is a Senior Student with The University of Arizona. He is with retail and his future plans include finding an internship in cybersecurity.



JAYDEN RYAN received the B.A.S. degree in cyber operations, with an emphasis in engineering from the University of Arizona, Tucson, AZ, USA, in 2025. His future plans include pursuing the M.S. degree in cybersecurity with the Daniel Felix School of Engineering and Computer Science, Denver, CO, USA.



MICAH STULL was born in Monticello, IN, USA. He received the A.A.S. degree (Hons.) from Cochise College, in 2023, and the B.A.S. degree (Hons.) in cyber operations with an engineering emphasis from The University of Arizona, in 2025. He was a United States Navy as a Sonar Technician (STG), completing four years of active service and three deployments. His research interests include offensive security automation using artificial intelligence (AI) and AI-augmented vulnerability assessments.



SHENGJIE XU (Senior Member, IEEE) received the M.S. degree in telecommunications from the University of Pittsburgh, and the Ph.D. degree in computer engineering from the University of Nebraska-Lincoln. He is currently an Assistant Professor with the College of Information Science at the University of Arizona. His research and educational projects have been supported by multiple U.S. federal funding agencies and Google. His research interests include cybersecurity, AI and machine learning for cybersecurity, and intelligent networking systems. He serves as a Technical Editor for IEEE Wireless Communications, where he was awarded the Exemplary Editor Award, and an Associate Technical Editor for *IEEE Communications Magazine*. He serves as a Symposium Co-Chair for the Communication and Information System Security Symposium at IEEE Global Communications Conference (GLOBECOM), in 2026. He holds multiple professional certifications in cybersecurity and computer networking.



PAUL WAGNER received the Ph.D. degree in cyber defense from Dakota State University. He is currently a Professor of practice in the cyber operations program with The University of Arizona. He is the Director of Arizona Cybersecurity Academy which focuses on K-20 education initiatives, workforce development, and community outreach and awareness. He is the Director of Arizona Cybersecurity Clinic which provides student experiential learning and cybersecurity support to government and community partners. Prior to working with The University of Arizona, he spent 20 years in the Army, where he developed his knowledge in computer networking and designing robust network architectures to support global operations in support of national and joint operations. In addition to his Ph.D. degree in cyber defense from Dakota State University, he holds over twenty technical certifications. His primary research interests include cybersecurity education, training, and workforce development.



ROBERT HONOMICHL received the B.S. degree in computer education and the M.S. degree in educational technology from Dakota State University, Madison, SD, USA, in 2001 and 2002, respectively, and the Ph.D. degree in technology innovation management, specializing in cybersecurity from Northcentral University, in 2023. He has over 25 years of experience teaching in higher and K-12 education, including positions with The University of Arizona, Tucson, AZ; Dakota State University, Madison, SD; and Madison Central School District, Madison, SD. He is currently an Assistant Professor of cyber operations with the College of Information Science, The University of Arizona, Tucson, AZ. His research interests include cybersecurity education, experiential learning, cybersecurity clinics, workforce development, risk assessment, and the human factors of cybersecurity.

STEVEN FORSYTH is currently a Cybersecurity Practitioner and an Educator with more than 40 years of experience in software engineering, secure systems design, and cyber operations. His research interests include applied cybersecurity, threat analysis, and translating industry practices into academic and experiential learning environments.